


# 软件工程基础

第6章 软件实现

杨溢龙

北航软件学院

# 目　录

Contents

## 软件工程基础

软件实现


Part 1 软件实现概论

Part 2 软件实现编程语言

Part 3 软件实现过程

Part 4 软件实现文档

北京航空航天大學
BEIHANG UNIVERSITY



Part 1

# 软件实现概论

# Part 1 软件实现概论

- 软件实现的概念
- 软件实现的任务
- 软件实现的特点与原则
- 软件实现的过程
- 软件实现的工具

# Part 1 软件实现概论

- 软件实现的概念
- 软件实现的任务
- 软件实现的特点与原则
- 软件实现的过程
- 软件实现的工具

# 何为软件实现

- 根据**软件设计模型**，编写出目标软件系统的**程序代码**，并对代码进行必要的**测试**，以发现和纠正代码存在中的**缺陷**，并将可运行的目标代码部署到目标计算机上运行

- 软件实现不仅要编写出程序代码，还要确保代码的质量，因此软件实现涉及多方面的开发工作，如**编码、测试、调试等**

# 软件实现概论

宏观上讲，“面向对象实现”的目标是：

- ✓**按照《详细设计说明书》的要求**，从公司的函数库、存储过程库、类库、构件库、中间件库中**挑选有关的部件**，
- ✓**遵照**软件公司的**程序设计规范**，**按照《详细设计说明书》**中对数据结构、算法分析和模块实现等**方面的设计说明**，
- ✓**用面向对象的语言**，通过穿针引线的方法，将这些部件组装起来，分别**实现各模块的功能**，从而**实现目标系统的功能、性能、接口、界面等要求**。

# 软件实现概论

- **微观上讲，软件实现**是指**通过编码、调试、单元测试、集成测试等活**动**创建软件产品**的过程。
- 软件实现与软件设计、软件测试密不可分。**软件设计为软件实现提供输入，软件实现的输出是软件测试的输入。**尽管软件设计和软件测试是独立的过程，但软件实现本身也涉及到设计和测试工作，它们之间的界限视具体项目而定。
- **软件实现**还会产生大量的**软件配置项**，例如，源文件、测试用例等，因此软件实现过程还涉及配置管理

# 软件实现与软件设计之间的关系

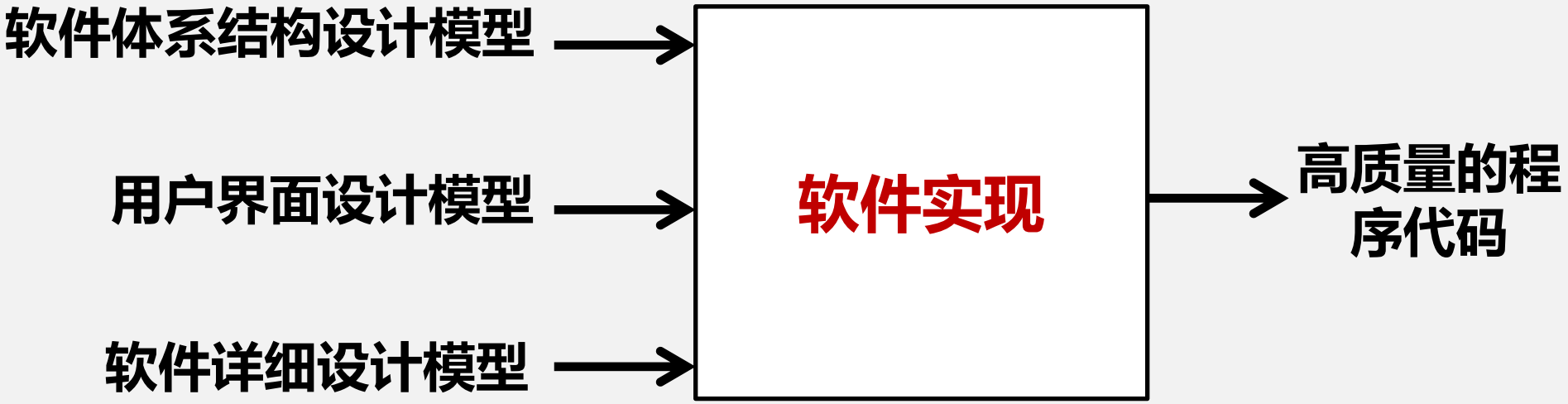
- **基于软件设计来开展软件实现**
  - ✓照软件**设计模型和文档**来进行编码

- **根据实现中发现的问题来纠正和完善软件设计**
  - ✓**设计不够详细**，程序员需要进行**进一步完善的软件设计**和程序设计，才能**编写出程序代码**
  - ✓**设计考虑不周全**，软件设计时没有认真考虑**编码实现的具体情况**（如程序设计语言和目标运行环境的选择），导致有些软件设计不能通过程序设计语言加以实现

# Part 1 软件实现概论

- 软件实现的概念
- 软件实现的任务
- 软件实现的特点与原则
- 软件实现的过程
- 软件实现的工具

# 软件实现的任务




- 软件实现的**输入**是**《详细设计说明书》**，**输出**是**源程序、目标程序及用户指南**
- 编程实现的主要方法是“**面向对象实现**”
- 因为现在流行的编程语言，基本上都是面向对象的语言，如Delphi，Visual Basic，C++，C#，Java，Python等。所以软件实现，主要的方法都是**“面向对象实现”**

# 软件实现的特点

## ❑生产性活动

- ✓需要根据**软件设计规格说明书和软件设计模型**，生产出与之相符合的软件制品，即程序代码
- ✓遵循设计文档和模型来编写程序，而且还要求程序员**遵循编码原则和风格**来编写出高质量的程序代码，并通过单元测试、集成测试、确认测试等一系列的软件测试活动来**保证代码质量**

## ❑创作性活动

- ✓发挥软件开发工程师的**智慧和主观能动性**，创作出目标软件系统的程序代码。这一过程高度依赖于程序员的**编程经验、程序设计技能和素养**，以及软件测试工程师的**软件测试水平**

# Part 1 软件实现概论

- 软件实现的概念
- 软件实现的任务
- 软件实现的原则
- 软件实现的过程
- 软件实现的工具

# 软件实现需考虑多方面的因素

## ❑与多类不同的人员相关

- ✓包括程序员、软件测试工程师等

## ❑**程序员**要考虑的因素

- ✓不仅要对照设计来编写代码，还需要通过遵循**编码规范**、**程序设计原则**等来**提高代码的质量**

## ❑**软件测试工程师**需要考虑的因素

- ✓针对代码开展测试，不仅要发现代码中存在的**功能性缺陷**，如代码功能实现不正确，还要发现代码中存在的**非功能性缺陷**

# 主观题 10分

设置

**思考和讨论：你是如何解决编码中遇到的问题？借助GPT来解决编程问题可信和可行吗？**



Answer

# 编写代码的原则（1/3）

## ❑易读，一看就懂

- ✓能够理解**代码的语义和内涵**，了解相关语句和代码的**实现意图**，**方便修改和维护**代码
- ✓采用缩进的方法来组织代码的显示，用括号来表示不同语句的优先级，对**关键语句、语句块、方法**等要**加以注释**

## ❑易改，便于维护

- ✓或者在适当的位置增加新的代码以完善代码功能，或者对某些代码进行修改以便纠正代码中的缺陷和错误
- ✓对将来可能需要进行修改和维护的代码（包括常元、变量、方法等）**进行单独的抽象、参数化和封装**，以便将来对其**修改时不会影响其他部分**的代码

# 编写代码的原则（2/3）

- **要有处理异常和提高代码的容错性**
  - ✓**编写必要的异常定义和处理代码**，使得程序能够对异常情况进行必要的处理，防止由于异常而导致的程序终止或崩溃
  - ✓编写程序代码以支持**故障检测、恢复和修复**，确保程序在**出现严重错误时仍然能够正常运行**，或者**当崩溃时能尽快恢复执行**
- **代码要与模型和文档相一致**
  - ✓程序员在编写代码的同时要同步修改和完善相应的软件设计模型和文档，**确保代码、模型和文档三者之间保持一致**

# 编写代码的原则（3/3）

- **降低代码的复杂度**
  - ✓将一个**类代码组织为一个文件**，并用**统一的命名规则**来命名文件
  - ✓在代码中适当的**增加注释**以加强对代码的理解，不用“goto”语句，**慎用嵌套或者减少嵌套的层数**，尽量选用简单的实现算法
- **尽可能地开展软件重用和编写可重用的程序代码**
  - ✓**尽可能地重用已有的软件制品**，如函数库、类库、软构件、开源软件、甚至代码片段等等
  - ✓在编码时要**考虑**所编写代码的**可重用性**，使得所编写的代码能为他人或者在其它软件系统开发中**被再次使用**

# 编程规范 (Style Guide)

- 遵循规范化的**代码布局**和**命名规范**，**易于理解的代码**
- 大型软件企业都有**一套开发标准、编码标准、界面标准**，在全企业内推广应用，形成自己的**企业文化**。新员工必须严格遵守，迅速适应

## Google Style Guides

Every major open-source project has its own style guide: a set of conventions (sometimes arbitrary) about how to write code for that project. It is much easier to understand a large codebase when all the code in it is in a consistent style.

"Style" covers a lot of ground, from "use camelCase for variable names" to "never use global variables" to "never use exceptions." This project (google/styleguide) links to the style guidelines we use for Google code. If you are modifying a project that originated at Google, you may be pointed to this page to see the style guides that apply to that project.

- AngularJS Style Guide
- Common Lisp Style Guide
- C++ Style Guide
- C# Style Guide
- Go Style Guide
- HTML/CSS Style Guide
- JavaScript Style Guide
- Java Style Guide
- Objective-C Style Guide
- Python Style Guide
- R Style Guide
- Shell Style Guide




https://google.github.io/styleguide/

## Table of Contents


- Preface
- 1. Programming Specification
  - Naming Conventions
  - Constant Conventions
  - Formatting Style
  - OOP Rules
  - Collection
  - Concurrency
  - Flow Control Statements
  - Code Comments
  - Other
- 2. Exception and Logs
  - Exception
  - Logs
- 3. MySQL Rules
  - Table Schema Rules
  - Index Rules
  - SQL Rules
  - ORM Rules





https://github.com/alibaba/Alibaba-Java-Coding-Guidelines

# 遵循编码风格（1/4）

- **格式化代码的布局，尽可能使其清晰、明了**
  - ✓充分利用**水平和垂直**两个方向的编程**空间**来组织程序代码，便于读者阅读代码
  - ✓适当地插入括号“{ }”，使语句的**层次性**、表达式运算次序等**更为清晰直观**
  - ✓有效地使用空格符，以显式地区别程序代码的**不同部分**（如程序与其注释）

```
package net.micode.notes.data;


import ...


public class NotesProvider extends ContentProvider {
    private static final UriMatcher mMatcher;

    private NotesDatabaseHelper mHelper;

    private static final String TAG = "NotesProvider";

    private static final int URI_NOTE            = 1;
    private static final int URI_NOTE_ITEM       = 2;
    private static final int URI_DATA            = 3;
    private static final int URI_DATA_ITEM       = 4;

    private static final int URI_SEARCH          = 5;
    private static final int URI_SEARCH_SUGGEST  = 6;

    static {
        mMatcher = new UriMatcher(UriMatcher.NO_MATCH);
        mMatcher.addURI(Notes.AUTHORITY, "note", URI_NOTE);
        mMatcher.addURI(Notes.AUTHORITY, "note/#", URI_NOTE_ITEM);
```

# 遵循编码风格（2/4）

- **尽可能提供简洁的代码，不要人为地增加代码的复杂度**
  - ✓使用**简单的数据结构**，避免使用难以理解和难以维护的数据结构（如多维数组、指针等）
  - ✓采用**简单而非复杂的实现算法**
  - ✓**简化程序中的算术和逻辑表达式**
  - ✓不要引入不必要的变元和动作
  - ✓防止变量名重载
  - ✓**避免模块的冗余和重复**

```
package net.micode.notes.data;


import ...


public class NotesProvider extends ContentProvider {
    private static final UriMatcher mMatcher;

    private NotesDatabaseHelper mHelper;

    private static final String TAG = "NotesProvider";

    private static final int URI_NOTE            = 1;
    private static final int URI_NOTE_ITEM       = 2;
    private static final int URI_DATA            = 3;
    private static final int URI_DATA_ITEM       = 4;

    private static final int URI_SEARCH          = 5;
    private static final int URI_SEARCH_SUGGEST  = 6;

    static {
        mMatcher = new UriMatcher(UriMatcher.NO_MATCH);
        mMatcher.addURI(Notes.AUTHORITY, "note", URI_NOTE);
        mMatcher.addURI(Notes.AUTHORITY, "note/#", URI_NOTE_ITEM);
```

# 遵循编码风格（3/4）

- 对代码辅之以**适当的文档，以加强程序的理解**
  - ✓有效、必要、简洁的**代码注释**
  - ✓代码注释的**可理解性、准确性和无二义性**
  - ✓确保**代码与设计**模型和文档的**一致性**

```
package net.micode.notes.data;


import ...


public class NotesProvider extends ContentProvider {
    private static final UriMatcher mMatcher;

    private NotesDatabaseHelper mHelper;

    private static final String TAG = "NotesProvider";

    private static final int URI_NOTE            = 1;
    private static final int URI_NOTE_ITEM       = 2;
    private static final int URI_DATA            = 3;
    private static final int URI_DATA_ITEM       = 4;

    private static final int URI_SEARCH          = 5;
    private static final int URI_SEARCH_SUGGEST  = 6;

    static {
        mMatcher = new UriMatcher(UriMatcher.NO_MATCH);
        mMatcher.addURI(Notes.AUTHORITY, "note", URI_NOTE);
        mMatcher.addURI(Notes.AUTHORITY, "note/#", URI_NOTE_ITEM);
```

# 遵循编码风格（4/4）

- 加强程序代码的结构化组织，提高代码的**可读性**
  - ✓按**一定的次序**来说明数据
  - ✓按字母顺序说明对象名
  - ✓**避免使用嵌套循环结构**和嵌套分支结构
  - ✓使用**统一的缩进规则**
  - ✓确保每个模块内部的**代码单入口、单出口**

```
package net.micode.notes.data;


import ...


public class NotesProvider extends ContentProvider {
    private static final UriMatcher mMatcher;

    private NotesDatabaseHelper mHelper;

    private static final String TAG = "NotesProvider";

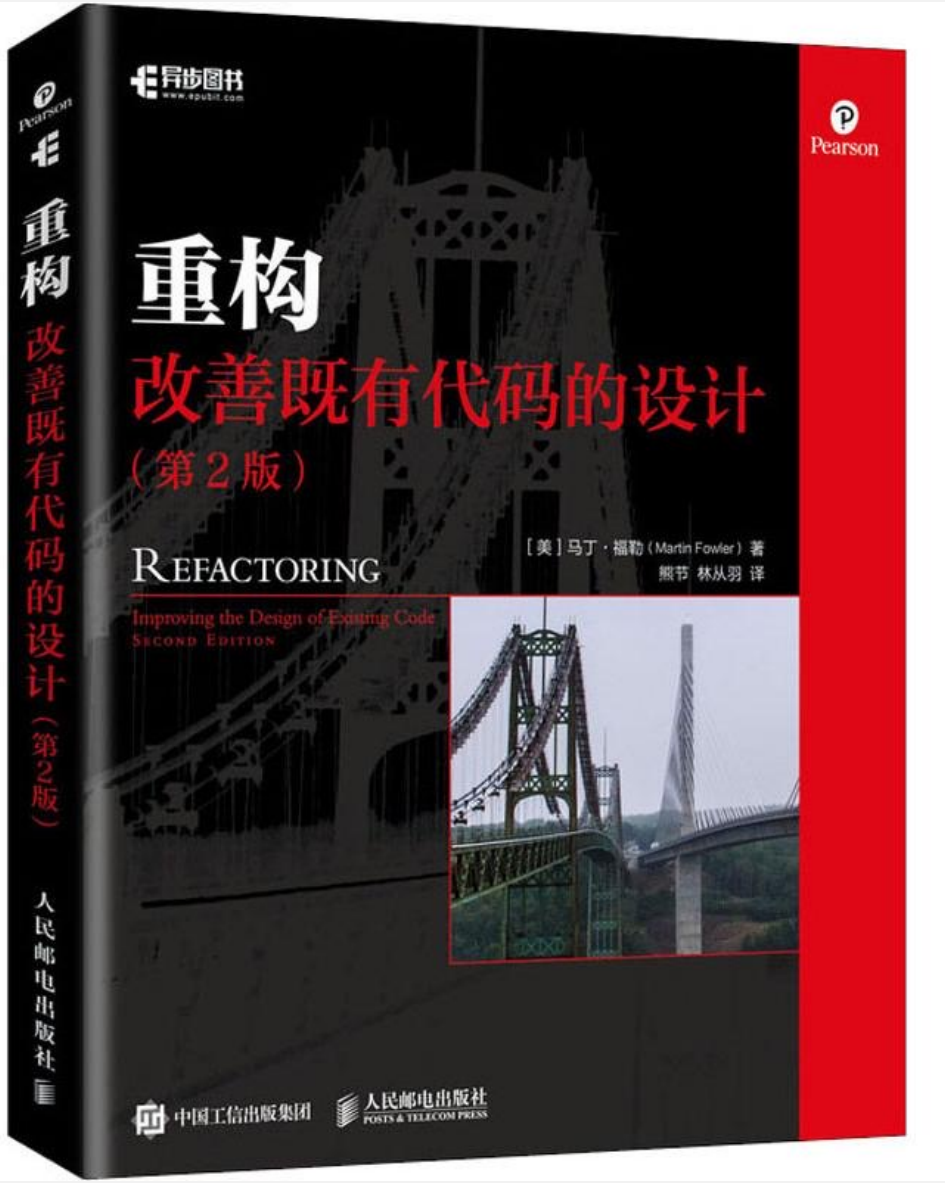
    private static final int URI_NOTE            = 1;
    private static final int URI_NOTE_ITEM       = 2;
    private static final int URI_DATA            = 3;
    private static final int URI_DATA_ITEM       = 4;

    private static final int URI_SEARCH          = 5;
    private static final int URI_SEARCH_SUGGEST  = 6;

    static {
        mMatcher = new UriMatcher(UriMatcher.NO_MATCH);
        mMatcher.addURI(Notes.AUTHORITY, "note", URI_NOTE);
        mMatcher.addURI(Notes.AUTHORITY, "note/#", URI_NOTE_ITEM);
```

# Just Read It...


代码大全
CODE COMPLETE
2
纪念版
Microsoft
A Practical Handbook of Software Construction


异步图书
Pearson
重构
改善既有代码的设计
（第2版）
REFACTORING
Improving the Design of Existing Code
SECOND EDITION
[美] 马丁·福勒（Martin Fowler）著
熊节 林从羽 译
人民邮电出版社
中国工信出版集团
人民邮电出版社

# 代码重构

代码重构是软件进化的重要手段，需要重构的例子有：

**(1)重复代码**：重复的代码需要做重复的修改，即要修改一个地方，就要平行地修改其他地方。

**(2)函数过长**：在面向对象的编程中，**很少需要超过一屏的函数**。出现这种情况，暗示着已使用了过程化编程方式，来进行面向对象的编程。

**(3)循环过长**：**过长的循环体最好变成独立的函数**，可有效地降低循环的复杂度。

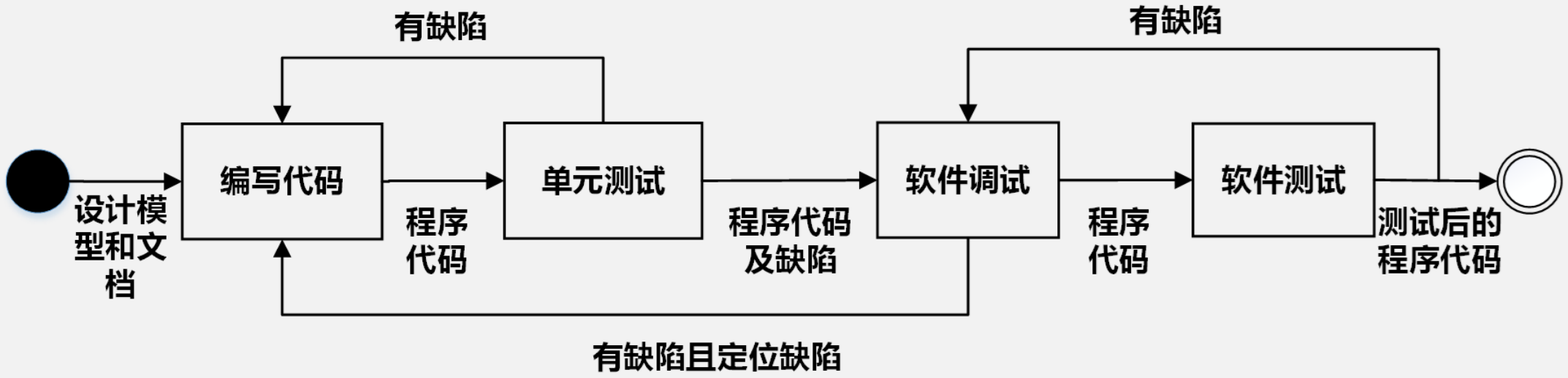
**(4)内聚性差**：如果发现**某个类是一些不相关的责任的集合**，这个类应该分解成多个类，每个类负责一个逻辑相关的责任集合。

**(5)参数过多**：好的函数通常比较简短，**不应有过多的参数**。过多的参数通常表示方法没有很好地抽象。

# Part 1 软件实现概论

- 软件实现的概念
- 软件实现的任务
- 软件实现的原则
- 软件实现的过程
- 软件实现的工具

# 软件实现的过程




**软件实现包含编码、测试、调试等一系列的开发活动**

# 编码+单元测试+调试

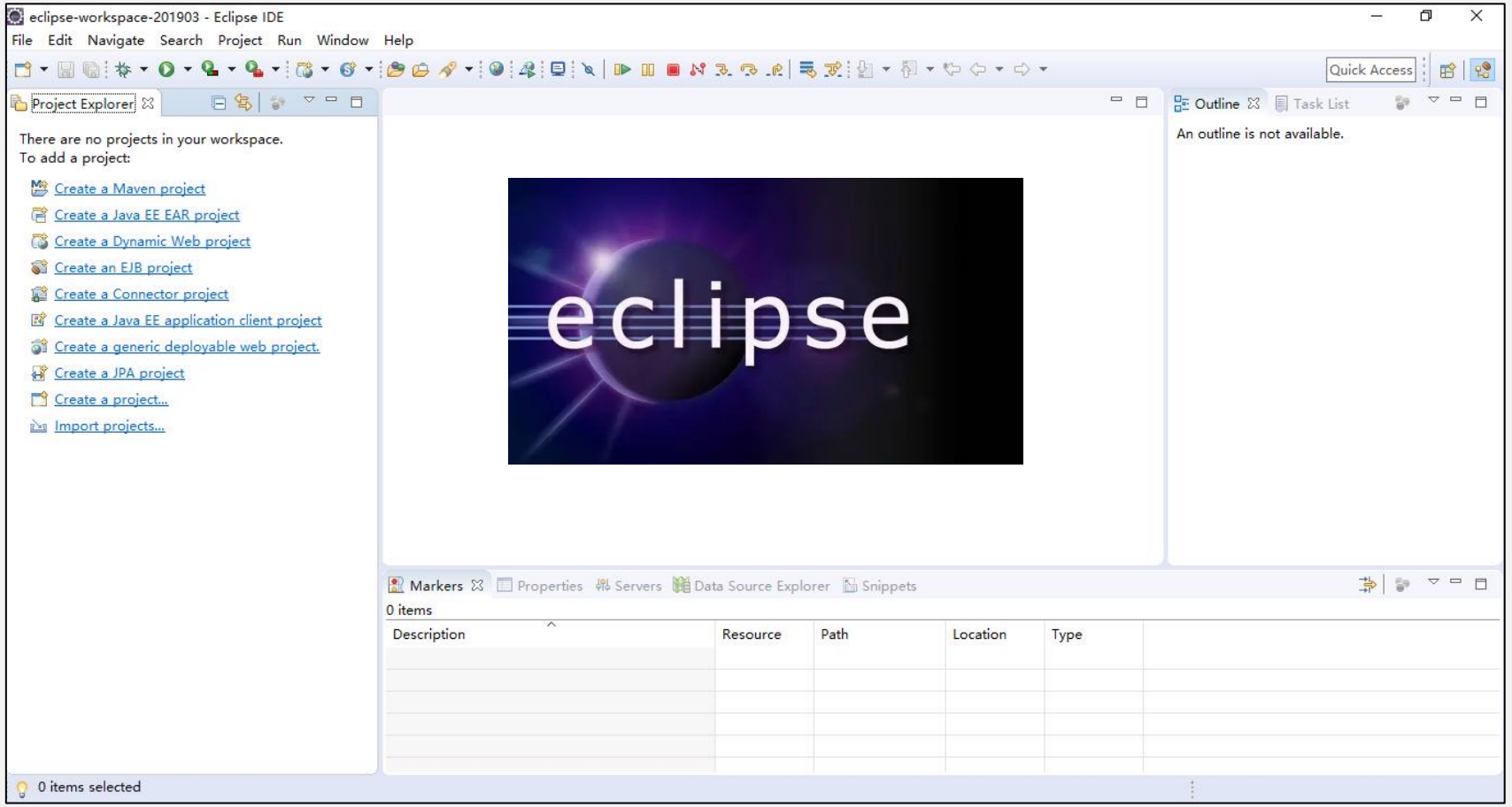
- 这三项工作均由程序员负责完成
- **编码**
  - ✓基于软件设计模型和文档，采用选定的程序设计语言，编写出目标软件系统的程序代码
- **单元测试**
  - ✓对自己编写的各个基本模块进行单元测试，以发现模块单元中存在的缺陷和问题
- **调试**
  - ✓发现产生缺陷原因，定位缺陷位置，进而对代码缺陷进行修复

# Part 1 软件实现概论

- 软件实现的概念
- 软件实现的任务
- 软件实现的原则
- 软件实现的过程
- 软件实现的工具

# 支持软件实现的CASE工具IDE

- ❑编辑器
- ❑编译器
- ❑调试器
- ❑测试工具
- ❑集成开发环境




Eclipse集合开发环境

# 目前主流IDE及其特点

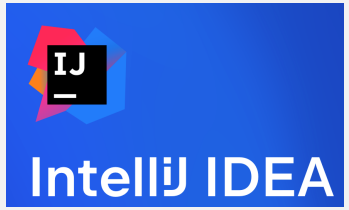
主流 IDE

## Visual Studio Code




支持各种开发语言

## JetBrains 系列




Java




Python




Rust

1. 轻量化、响应式
**用户体验极佳**

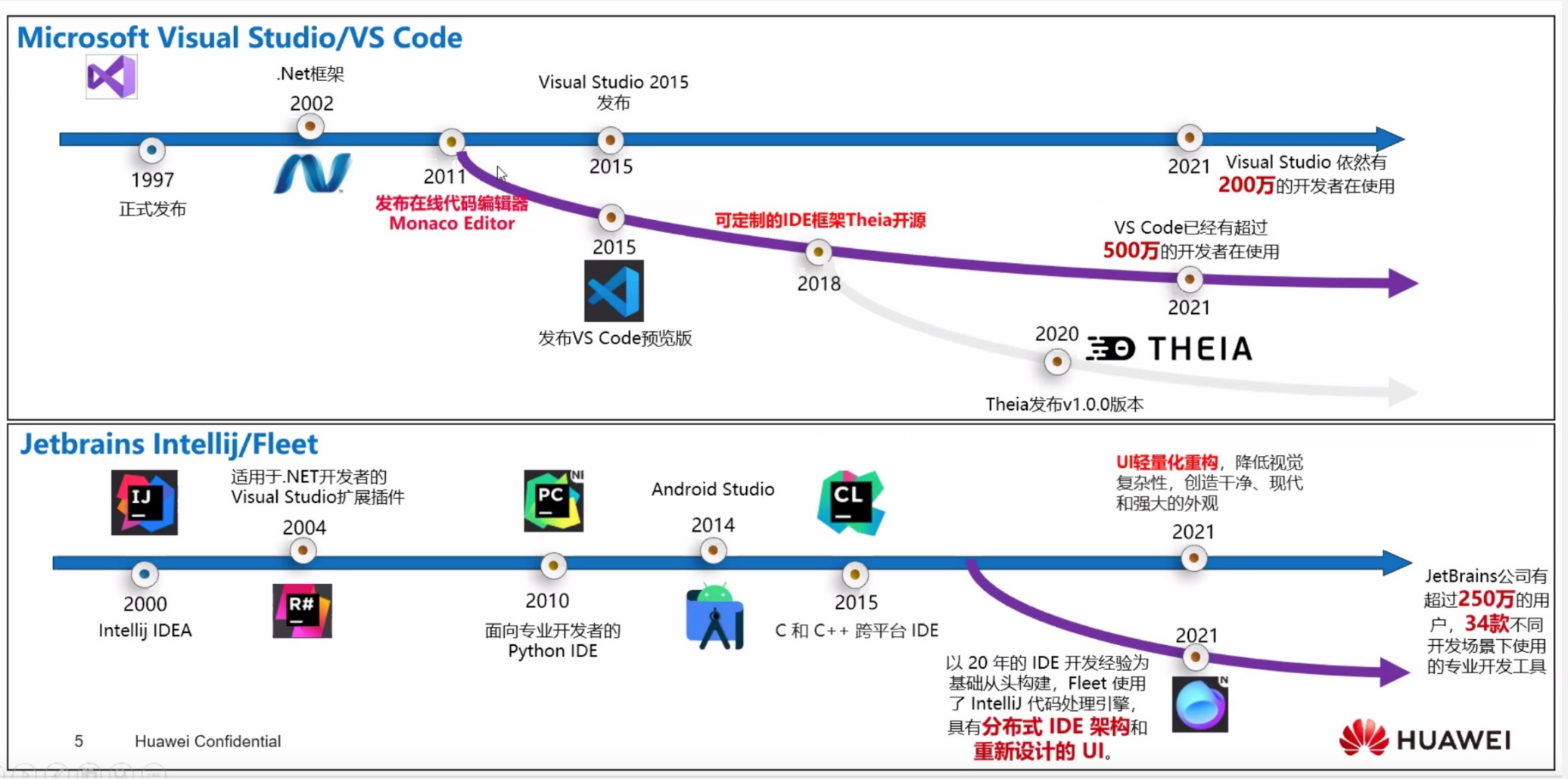
2. 各种开发框架高度集成
**入门门槛极低**

3. 代码库、CI/CD及云端环境高度集成
**快速迭代与部署**

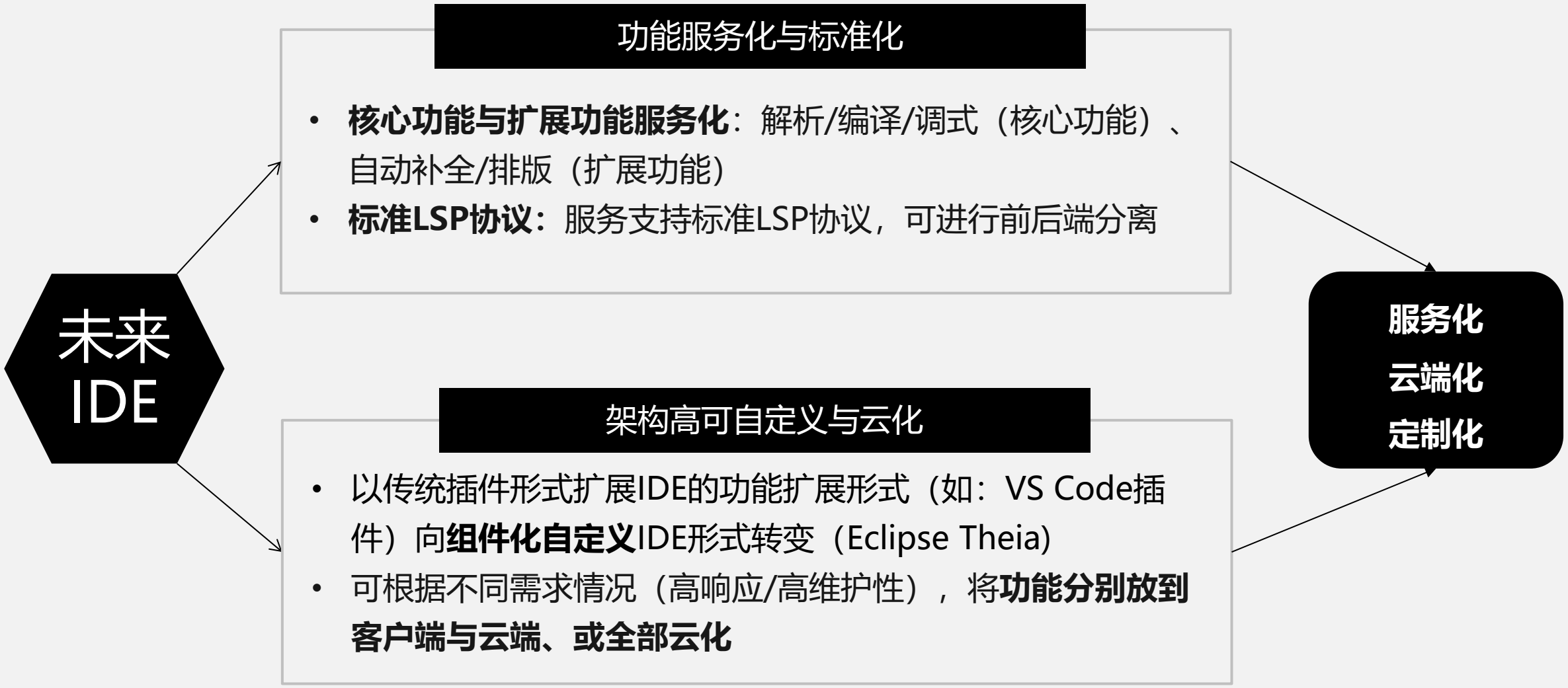
4. 支持自定义插件
**高扩展性**

# 目前主流IDE发展历史

主流 IDE



# 未来IDE特点与趋势


未来IDE
功能服务化与标准化
• 核心功能与扩展功能服务化：解析/编译/调式（核心功能）、自动补全/排版（扩展功能）
• 标准LSP协议：服务支持标准LSP协议，可进行前后端分离
架构高可自定义与云化
• 以传统插件形式扩展IDE的功能扩展形式（如：VS Code插件）向组件化自定义IDE形式转变（Eclipse Theia）
• 可根据不同需求情况（高响应/高维护性），将功能分别放到客户端与云端、或全部云化
服务化
云端化
定制化

北京航空航天大學
BEIHANG UNIVERSITY



Part 2

# 编程语言

# Part 2 编程语言

- 编程语言类别
- 编程语言举例
- 编程语言排名
- 编程语言选择

# Part 2 编程语言

- 编程语言类别
- 编程语言举例
- 编程语言排名
- 编程语言选择

# 程序设计语言的类别（1/3）

## ❑机器语言

- ✓由“0”、“1”所组成的机器指令
- ✓极为繁琐、费时费力的工作；软件开发效率非常低，而且程序代码可读性非常差，极容易出错，**不易于维护、移植性差**；但程序代码的**执行效率会非常高**

## ❑汇编语言

- ✓一种**低级语言**，用**助记符**代替**机器指令的操作码**，用地址符号或标号代替指令或操作数的地址
- ✓较为低级和复杂，程序**可读性差，代码编写的效率低**，对代码进行**维护非常困难**，**程序调试也不容易**，代码兼容性差
- ✓程序代码占用**存储空间少、运行速度快、执行效率高**

# 程序设计语言的类别（2/3）

## ❑结构化程序设计语言

- ✓以**过程或函数**作为基本的编程单元，采用**三类控制结构**（顺序、条件和循环）来刻画模块的处理过程和流程
- ✓属于**高级程序设计语言**，程序可读性、可理解性、可维护性等有了**明显的提升**；配套**CASE工具较为完善**，有结构化程序设计方法学的指导
- ✓**不足**：以**过程和函数作为基本模块**，模块的粒度小，可重用性差；程序代码抽象层次低，无法对问题域及其求解**进行自然抽象**
- ✓如C、Fortran、Pascal等

# 程序设计语言的类别（3/3）

## ❑面向对象程序设计语言

- ✓以类作为基本的模块单元，借助于**面向对象的一组概念和机制**来进行程序设计，
- ✓有系统的**方法学指导**，建立起可**直观反映问题域**、模块粒度更大、**可重用性更好**的程序代码，已经成为计算机领域的**主流编程语言**
- ✓如Java、C++等

## ❑描述性程序设计语言

- ✓描述程序需要**解决什么样的问题**，无需在程序中显式地**定义如何来解决问题**
- ✓如Prolog、Lisp、ML等

# 程序设计语言的表达能力

| 编程语言的类别 | 平均代码量 | 编程语言 | 平均代码量 |
| --- | --- | --- | --- |
| 机器语言 | 320 | C | 128 |
| 汇编语言 | 107 | Fortran | 107 |
| 高级语言 | 80 | C++/Java | 53 |

**一个功能点用不同的语言来实现所需的平均代码量**

# 编程语言

编程语言可分为三类：

1. 命令式语言（Imperative Language): C/C++/Java/Python
2. 函数式语言（Functional Language): Lisp / MK
3. 逻辑语言（Logic Language）： Prolog / ASP(Answer Set)

# Part 2 编程语言

- 编程语言类别
- 编程语言举例
- 编程语言排名
- 编程语言选择

# C/C++语言

- 面向过程、通用的结构化（面向对象）程序设计语言
- 简洁的语言
- 结构化的控制语句
- 丰富的数据类型和运算符
- 良好的可移植性
- 目标代码执行效率高

# Java语言

- 纯粹的面向对象编程语言
- 简单性
- 分布性
- 兼具编译和解释性以及可移植性
- 强类型语言和健壮性
- 安全性，没有指针，使用字节码验证策略，防止恶意代码

# Python语言

- 面向对象、解释型、通用的脚本编程语言
- 功能强大，Python 类库极其丰富
- 语法简单，入门容易
- 开源和免费，可移植性
- 解释性
- 混合型语言，既支持面向过程的编程也支持面向对象的编程

# 编程语言举例

- Swift / Kotlin / ArkTS
- TypeScript / Rust / 仓颉

# Part 2 编程语言

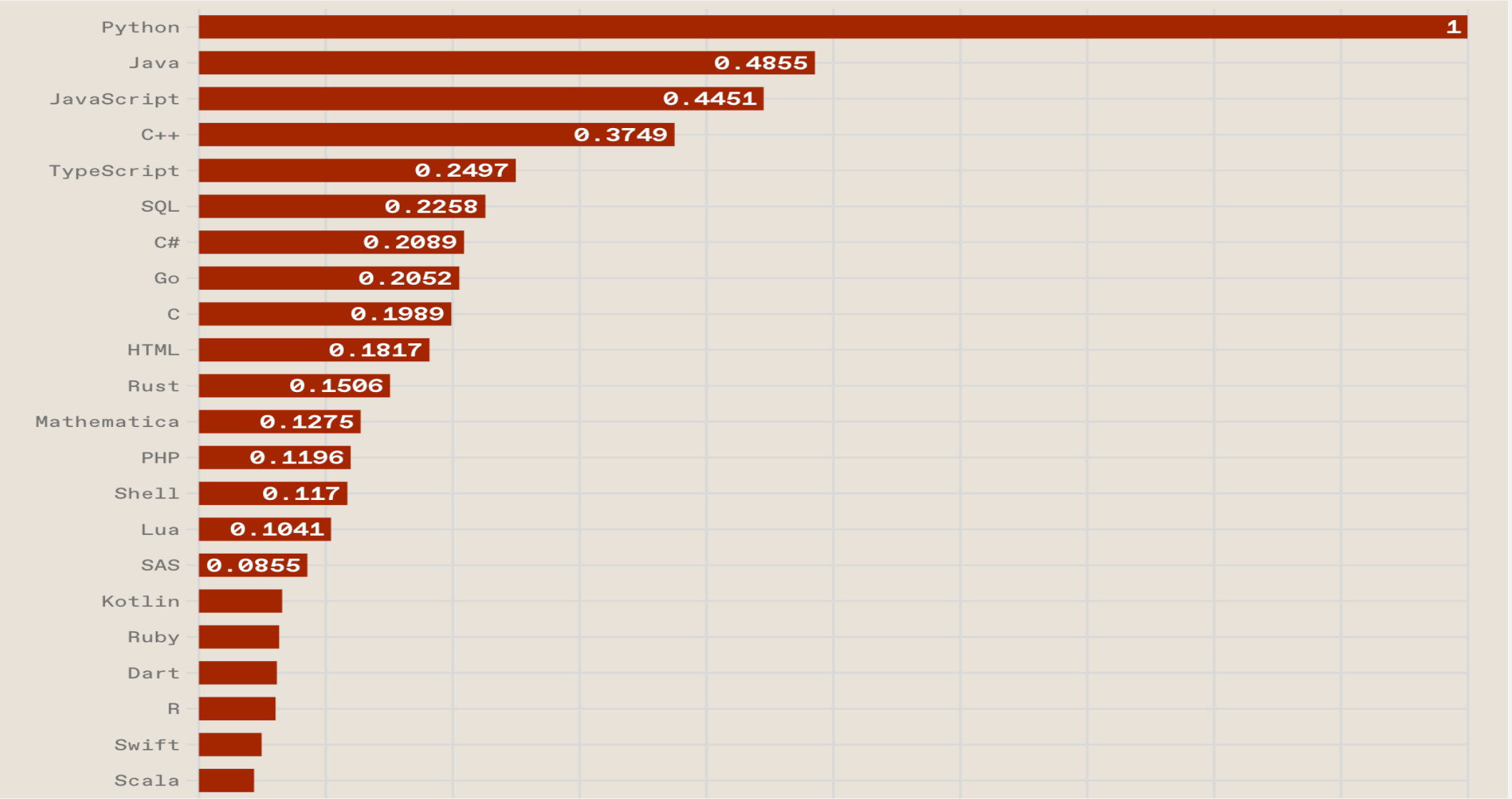
- 编程语言类别
- 编程语言举例
- 编程语言排名
- 编程语言选择

# 不同编程语言 Top 10

| May 2025 | May 2024 | Change | Programming Language | Ratings | Change |
|---|---|---|---|---|---|
| **1** | **1** | | **Python** | **25.35%** | **+9.02%** |
| 2 | 3 | ↑ | C++ | 9.94% | +0.41% |
| 3 | 2 | ↓ | C | 9.71% | -0.27% |
| 4 | 4 | | Java | 9.31% | +0.62% |
| 5 | 5 | | C# | 4.22% | -2.27% |
| 6 | 6 | | JavaScript | 3.68% | +0.66% |
| 7 | 8 | ↑ | Go | 2.70% | +1.10% |
| 8 | 7 | ↓ | Visual Basic | 2.62% | +0.61% |
| 9 | 11 | ↑ | Delphi/Object Pascal | 2.29% | +1.05% |
| 10 | 9 | ↓ | SQL | 1.90% | +0.45% |

来源： https://www.tiobe.com/tiobe-index/

# 不同编程语言 Top 10




来源：https://spectrum.ieee.org/the-top-programming-languages-2023

# Part 2 编程语言

- 编程语言类别
- 编程语言举例
- 编程语言排名
- 编程语言选择

# 程序设计语言的选择（1/3）

## ❑软件的应用领域

- ✓不同应用领域的软件通常会选择不同的程序设计语言来加以实现
- ✓科学和工程计算领域选用Fortran、C等程序设计语言，数据库应用软件开发会选用Delphi、Visual Basic、SQL等程序设计语言，机器人等嵌入式应用选用C、C++、Python等程序设计语言，互联网应用开发选用Java、ASP等程序设计语言

## ❑与遗留软件系统的交互

- ✓考虑待开发软件系统是否需要与遗留软件系统存在交互。如果有该方面的实际需要，那么程序员需要解决二个系统之间的互操作问题

# 程序设计语言的选择（2/3）

- **软件的特殊功能及需求**
  - ✓**是否需要与底层的硬件系统**进行交互，如果需要，可以考虑采用诸如C、汇编语言
  - ✓**是否需要丰富的软件库来支持功能**的实现，如果需要，可以考虑具有丰富软件库的编程语言，如Python、Java等
  - ✓是否需要对相关的**知识进行表示和推理**，如果需要，可以考虑选用描述性的程序设计语言，如Prolog、Lisp等

# 程序设计语言的选择（3/3）

## ❑软件的目标平台

- ✓如果目标软件系统需要运行在特定的**软件开发框架**、**软件中间件**、**基础设施之上**，那么程序员还需要考虑目标平台对程序设计语言的支持，并依此来选定所需的编程语言
- ✓如果目标软件系统需要部署在**Java EE架构**之上，那么就需要选择**Java编程语言**；如果需要借助于**ROS来开发机器人软件**，那么建议选择**C**、**C++**和Python等编程语言

## ❑程序员的编程经验

- ✓应该选择对于自己而言**较为熟悉的语言**，尽量避免选择没有使用过的程序设计语言

# 主观题 10分

设置

**编程实现面临的主要挑战是什么？它对程序员和测试工程师提出什么样的要求?**



Answer

北京航空航天大學
BEIHANG UNIVERSITY



Part 3

# 软件编码过程

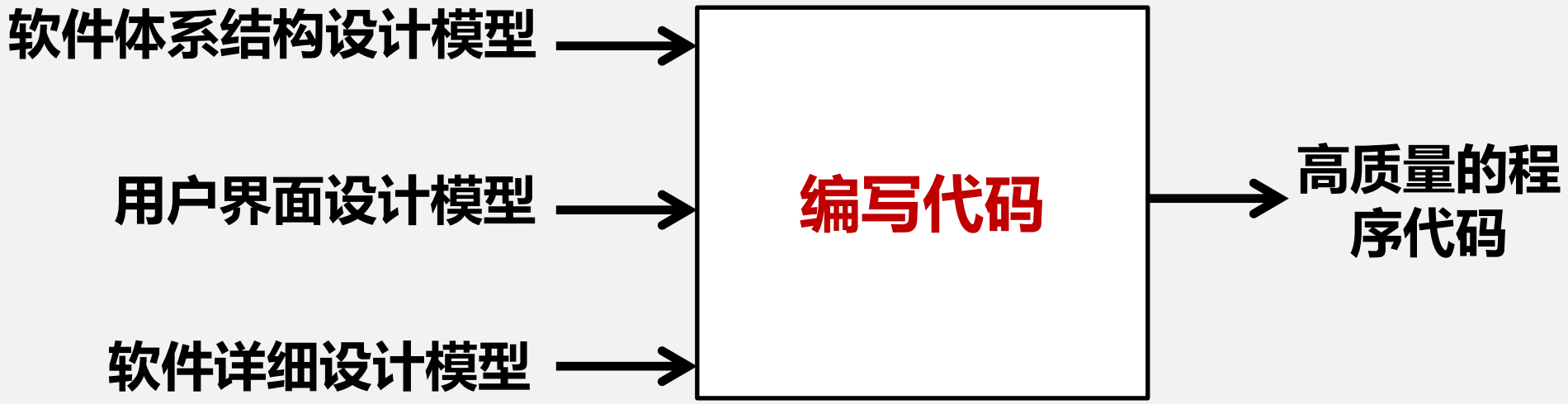
# Part 3 软件编码过程

- 编写代码的任务
- 编程类的代码
- 编程界面代码
- 编程数据代码
- 基于框架编写代码

# Part 3 编码技术

- 编写代码的任务
- 编程类的代码
- 编程界面代码
- 编程数据代码
- 基于框架编写代码

# 编写代码的任务




- **根据软件设计信息，借助于程序设计语言，编写出目标软件系统的源程序代码，开展程序单元测试、代码审查等质量保证工作**
  - ✓编写代码既是**一个生成代码的过程**，也是对生成的**代码进行质量保证**的过程

# Part 3 编码技术

- 编写代码的任务
- 编程类的代码
- 编程界面代码
- 编程数据代码
- 基于框架编写代码

# 编写类代码

① 编写实现类的代码
② 编写实现类方法的代码
③ 编写实现类间关联的代码
④ 编写实现设计类间聚合和组合关系的代码
⑤ 编写实现接口关系的代码
⑥ 编写实现继承关系的程序代码
⑦ 编写实现包的代码

# （1）编写实现类的代码

- 设计模型（如设计类图）详细描述了软件系统中类的详细设计信息，包括可见性、类名、属性、方法等等
- 程序员需要将这些设计信息直接转换为用程序设计语言表示的实现结构和代码

```
public class User {
    private String account; //用户的账号
    private String password; //用户的密码
    private String name; //用户的名字
    private String mobile; //用户的移动手机号
    private int type; //用户的类别

    public void User(String account, String password); //构造函数
    public void User(String account, String password,
                     String name, String mobile, int type);
    public String getUserName( ); //获取用户的名字
    public String getUserAccount( ); //获取用户的账号
    public int getUserType( ); //获取用户的类别
    public String getUserMobile( ); //获取用户的手机号
    public void setUserPsw(String userPsw); //设置用户的密码
}
```

# （2）编写实现类方法的代码

- 基于**类方法**的设计描述（UML的**活动图**表示），程序员可以依此为依据来编写类方法的实现代码

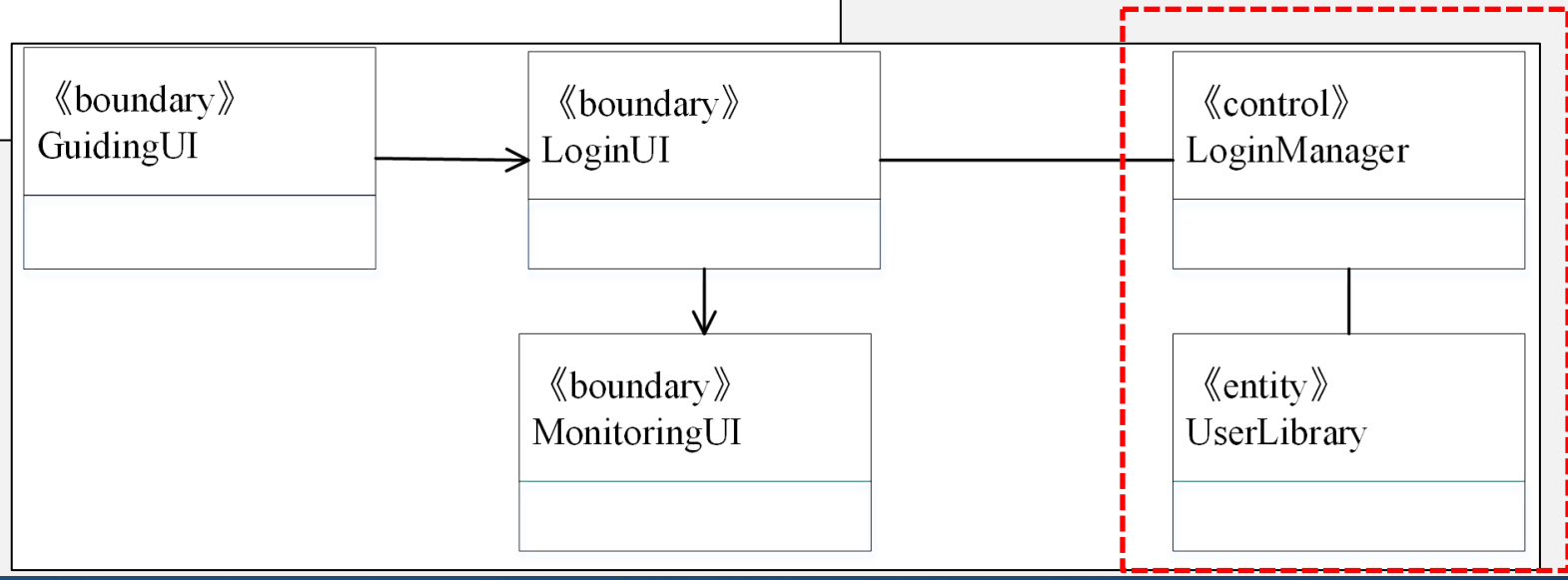
```
public int login(String account, String password) {
    final int ERROR_ACCOUNT_EMPTY = 1;    //表示账号为空的错误代码
    final int ERROR_PASSWORD_EMPTY = 2;    //表示密码为空的错误代码
    final int ERROR_INVALID_USER = 3;    //表示用户非法的错误代码
    final int LOGIN_SUCCESS = 0;    //表示用户合法的代码
    int result ;

    If (account.getLength() = = 0) {      //检查 account 是否为空串
        result = ERROR_ACCOUNT_EMPTY; //表示账号为空
    } else if (password.getLength() = = 0)    { //检查 password 是否为空串
        result =ERROR_PASSWORD_EMPTY; //表示密码为空
    } else {
            //向 UserLibrary 对象发消息以验证用户的身份是否合法
            boolean validUser = userLib.isUserValid(account, password);
            If (validUser) {
                result = LOGIN_SUCCESS;
            } else {
                result =ERROR_INVALID_USER;
            }
    }
    return result;
}
```

# （3）编写实现类间关联的代码

- 将类间关联关系的**语义信息**具体落实到相应类的**程序代码中**，即综合考虑关联关系的**方向性、多重性、角色名和约束特性**等信息来编写相关的类程序代码

```
public class LoginManager {
    private UserLibrary userLib; //UserLibrary 的对象
    ...
}
```



# （4）编写实现设计类间聚合和组合关系的代码

- 可以采用类似于实现**关联关系的方法**来编写实现聚合和组合关系的代码
- 根据多重性来设计相应类属性的数据结构

# （5）编写实现接口关系的代码

- 类设计模型可能包含有表征**类与接口之间实现关系**的语义信息
- 诸多面向对象程序设计语言（如Java、C++等）提供了专门针对接口实现的语言机制，因而可以直接将**接口设计信息转换为相应的程序代码**
  - ✓如“Implement”机制

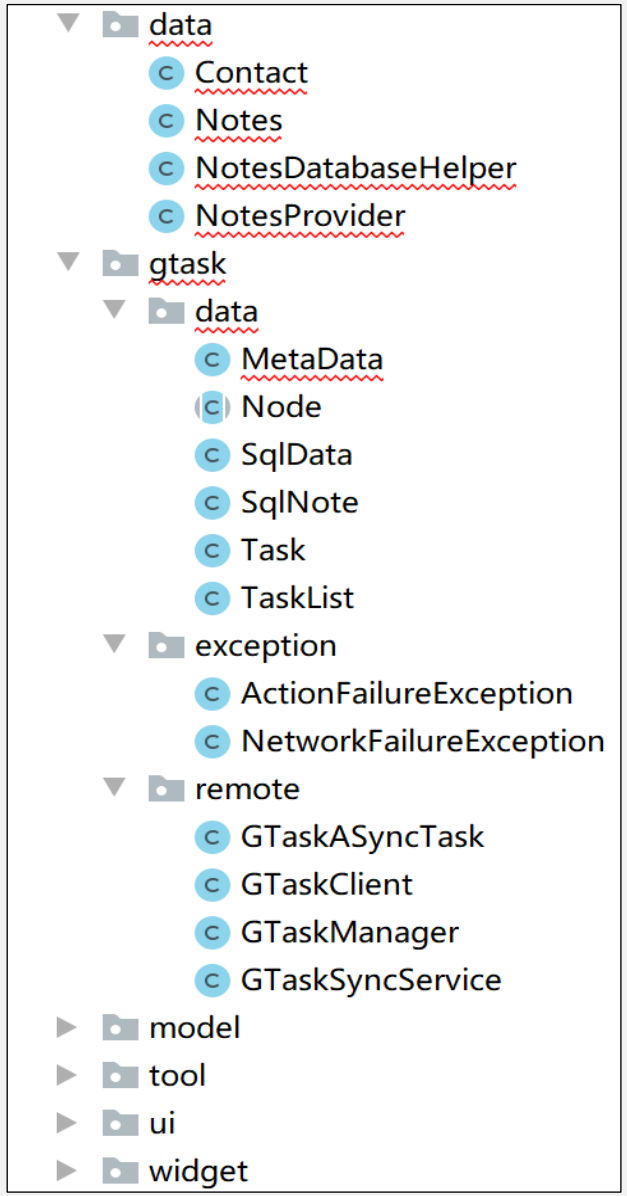
# （6）编写实现继承关系的程序代码

- 面向对象程序设计语言（如Java、C++）提供了继承机制以及相应的语言设施
  - ✓Java支持单重继承，C++支持多重继承
  - ✓如“extends”机制
- 将设计模型中的类间继承关系用程序设计语言提供的语言机制来表示

```
public class LoginUI extends Activity {
        //成员方法说明
        public void login();
        public void cancel();
        public boolean isInputAccountValid();
        public boolean isInputPswValid();
}
```

# （7）编写实现包的代码

- 用包（package）来组织和管理软件系统中的类
- 包是对软件系统中模块的逻辑划分，也可以将包视为是一种子系统
- 面向对象程序设计语言（如Java）提供了对包进行编程的语言机制，每个包对应于代码目录结构中的某个目录



# Part 3 编码技术

- 编写代码的任务
- 编程类的代码
- 编程界面代码
- 编程数据代码
- 基于框架编写代码

# 编写用户界面代码

## ❑用户界面设计模型

- ✓描述了构成用户界面的各个界面设计元素（包括静态元素、动态元素、用户输入元素、用户命令元素等）
- ✓用户界面之间的跳转关系

## ❑编码实现

- ✓编写界面类属性的代码以定义界面设计元素
- ✓编写界面类的方法以对界面操作或者对界面事件进行响应处理

# 示例：“LoginUI”的编码实现




```
public class LoginUI extends Activity {
    private EditText mAccount;
    private EditText mPsw;
    private Button mCancelButton;
    private Button mLoginButton;
    private UserLibrary mUserLibrary;

    public void onCreate(Bundle savedInstanceState) {
        ...
    }
    //用户登录方法
    public void login() {
        ...
    }
    //取消登录方法
    public void cancel() {
        ...
    }
```

# 示例：“LoginUI”的编码实现




```
OnClickListener mListener = new OnClickListener() {
    public void onClick(View v) {
        switch (v.getId()) {
            case R.id.login_btn_cancel: //取消
                onPause();
                break;
            case R.id.login_btn_login: //确认
                login();
                break;
        }
    }
};
```

# Part 3 编码技术

- 编写代码的任务
- 编程类的代码
- 编程界面代码
- 编程数据代码
- 基于框架编写代码

# 编写数据设计代码

## ❑数据设计

- ✓定义了软件系统中需要**持久保存数据及其组织**（如数据库的表、字段）和**存储**（如数据库中的记录）**方式**
- ✓设计了**相应的类及其方法来读取、保存、更新和查询持久数据**

## ❑编码实现

- ✓创建相应的**数据库关系表格**及其**内部的各个字段选项**等，确保它们满足设计的要求和约束
- ✓**编写相应的程序代码来操作数据库**，如增加、删除、更改、查询数据记录等

# 示例：“T_User”表的创建

```
private static final String TABLE_NAME = "T_User";
public static final String USER_ACCOUNT = "user_account";
public static final String USER_NAME = "user_name";
public static final String USER_PSW = "user_password";
public static final String USER_MOBILE = "user_mobile";
public static final int USER_TYPE= "user_type";

//创建数据库表“T_User”的 SQL 语句
String DB_CREATE = "CREATE TABLE " + TABLE_NAME + " ("
        + USER_ACCOUNT + " varchar primary key," + USER_NAME + " varchar,"
        + USER_PSW + " varchar," + USER_MOBILE + " varchar,"
        + USER_TYPE + " integer" + ");";
db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME + ";"); //执行 SQL 语句
db.execSQL(DB_CREATE);
```

# 示例：连接和关闭数据库的代码

```
// 打开数据库
public void openDataBase() throws SQLException {
        mDatabaseHelper = new DataBaseManagementHelper(mContext);
        mSQLiteDatabase = mDatabaseHelper.getWritableDatabase();
}
// 关闭数据库
public void closeDataBase() throws SQLException {
        mDatabaseHelper.close();
}
```

# 示例：操作数据库的程序代码

```
//向数据库表中插入用户的数据
public booean insertUser(User user) {
        String UserAccount = user.getUserAccount();
        String UserName=user.getUserName();
        String UserPsw=user.getUserPsw();
        String UserMobile=user.getUserMobile();
        int UserType=user.getUserType();
        ContentValues values = new ContentValues();
        values.put(USER_NAME, UserName);
        values.put(USER_PSW, UserPsw);
        values.put(USER_MOBILE, UserMobile);
        values.put(USER_TYPE, UserType);
        return mSQLiteDatabase.insert(TABLE_NAME, ID + "=" + UserAccount , values);
}
```

# 示例：判断用户账号和密码合法性的代码

```
// 基于账号和密码来判断用户身份的合法性
public boolean verifyUserValidity(String account,String psw){
        boolean result= FALSE;
        Cursor mCursor=mSQLiteDatabase.query(TABLE_NAME, null,
USER_ID+"="+account+" and "+USER_PSW+"="+psw, null, null,
null, null);
        if(mCursor!=null){
                result=TRUE;
                mCursor.close();
        }
        return result;
}
```

# Part 3 编码技术

- 编写代码的任务
- 编程类的代码
- 编程界面代码
- 编程数据代码
- 基于框架编写代码

# So Far So Good?

学会了如何 coding…
学会了复用代码片段…
是否够用?

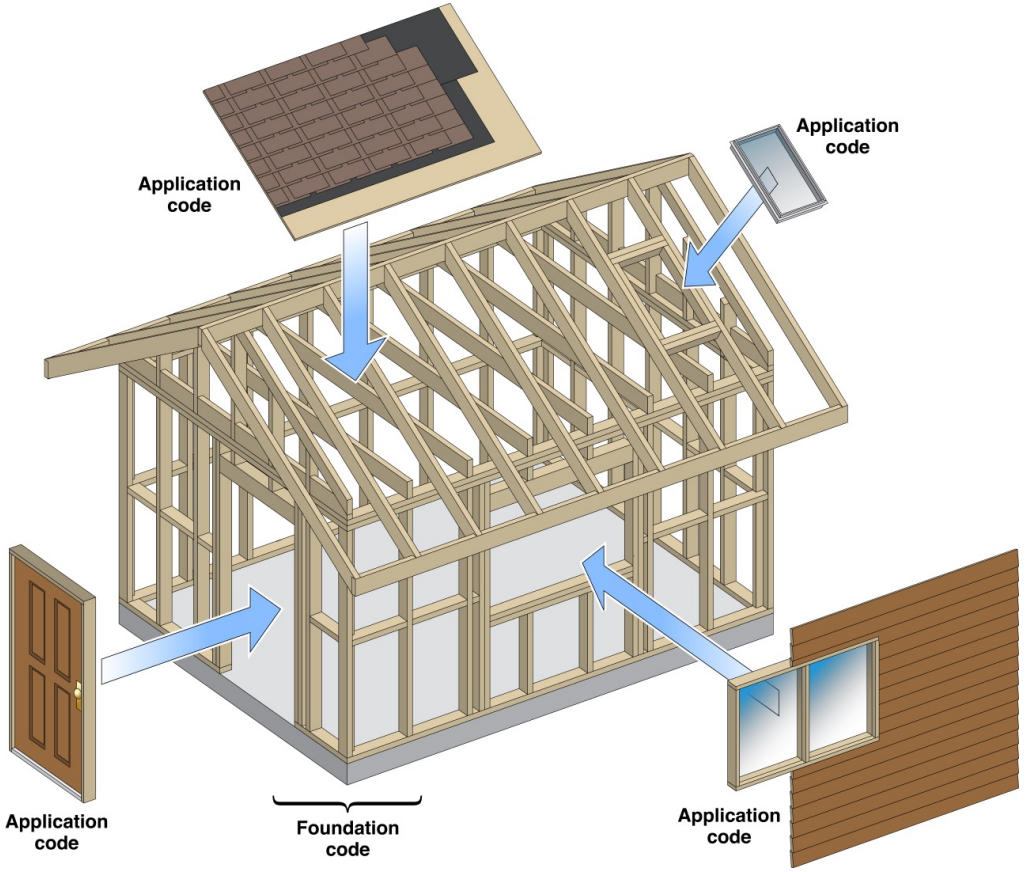
是否应该要从头开始实现 (start from scratch)?

**Absolutely No!**
**用好框架…**

# 什么是框架?

框架：建筑工程中由梁、柱等联结而成的结构



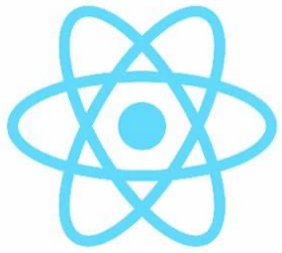



来源：https://www.bigstockphoto.com/image-2676682/stock-photo-house-framework

# 什么是框架?

- A framework is a **pre-defined foundation structure** that you can build software upon, so you're not starting entirely from scratch
- A framework is **a set of programming tools** (pre-written codes, libraries, security features,MVC architecture, etc.) to build **well-structured, reliable** software and systems

可被应用开发者定制的**应用骨架**，是实现某种功能的半成品，提供了一些**常用工具类**和一些**基础通用化组件**，可以供开发人员在此基础上，更高效的满足各自的业务需求











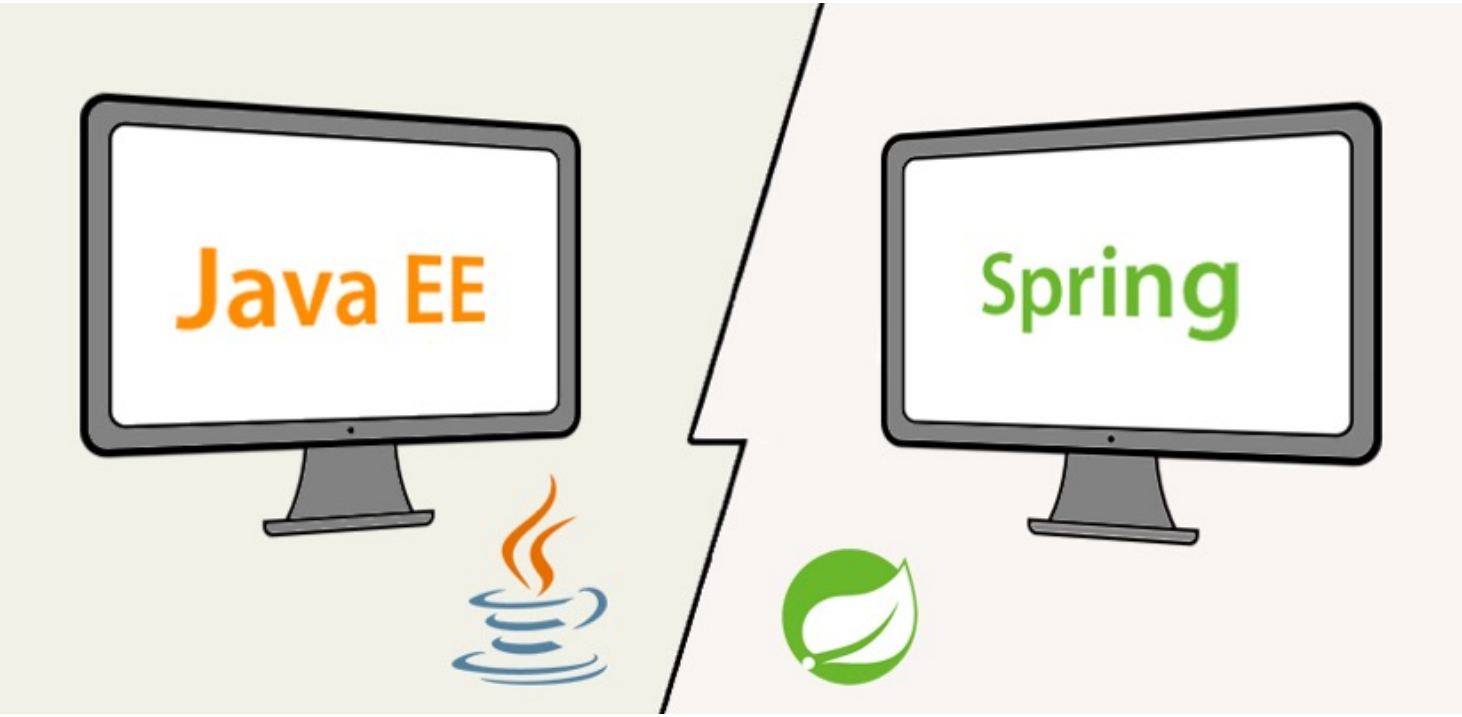







# Case Study: 基于Java技术栈的系统实现

- 例1：基于Java EE的单体式应用实现
- 例2：基于 SpringCloud的微服务架构应用实现

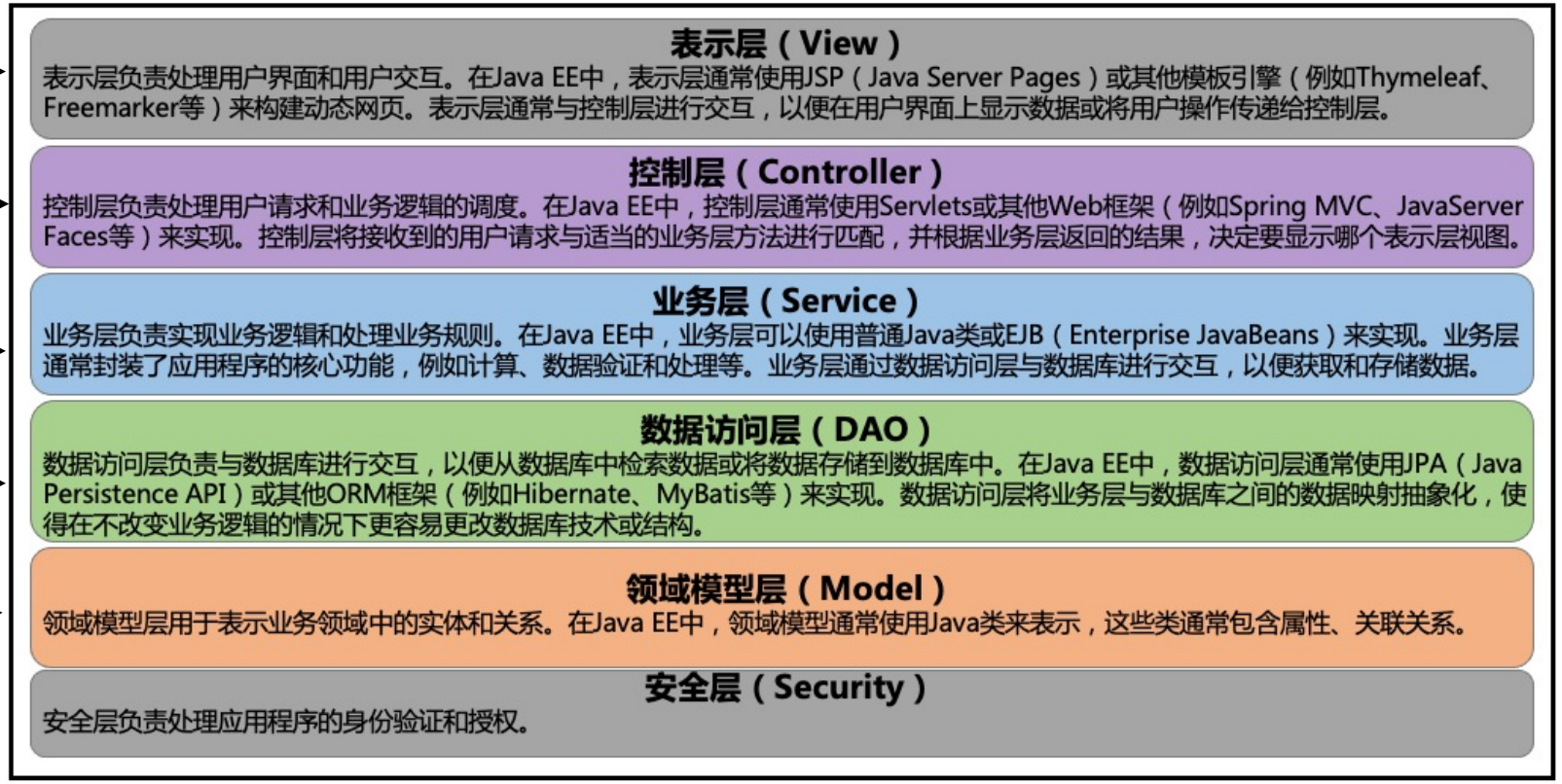


# 例1：基于Java EE的单体式应用实现

## 分层OO设计模型

- ✓ View （用户视图）
- ✓ Controller（控制器）
- ✓ Service（业务逻辑）
- ✓ EntityManager（实体管理）
- ✓ Entity （实体类）

## Java EE分层架构

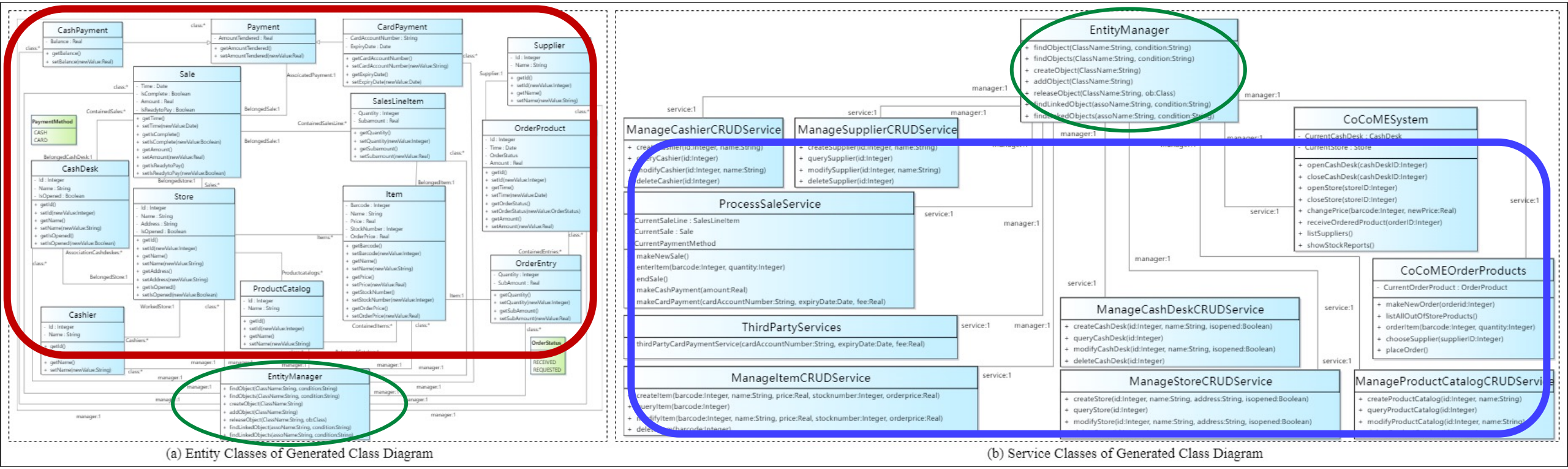


# 面向对象的设计模型 – 类图

- 类图描述系统的静态结构

**实体类（及关系）**

**EntityManager（管理对象）**




(a) Entity Classes of Generated Class Diagram

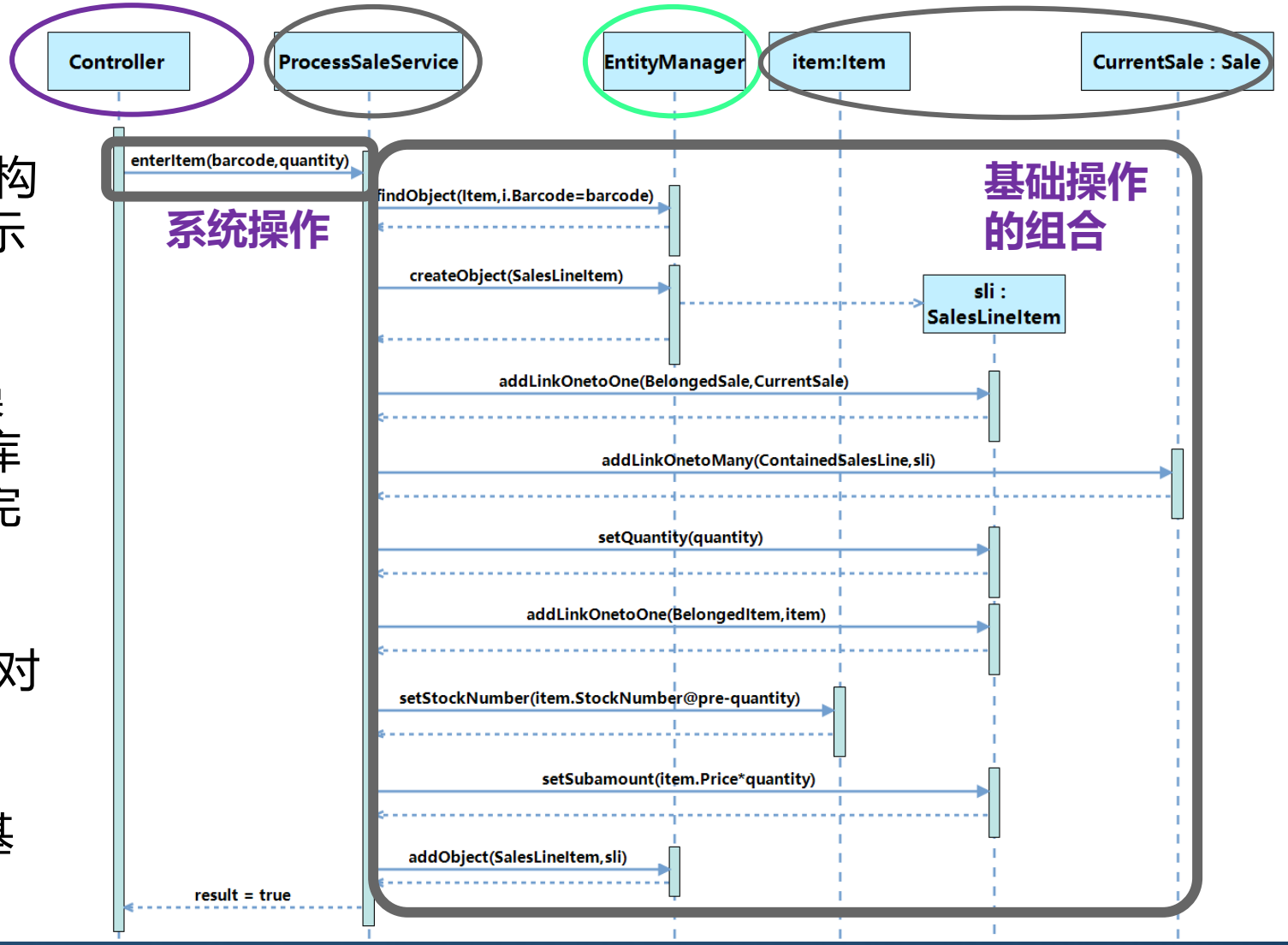
(b) Service Classes of Generated Class Diagram

**Service（整合业务逻辑）**

# 面向对象的设计模型 – 顺序图

- **顺序图**：描述系统的动态行为，主要是描述系统内的对象如何通过基础操作的协作完成复杂的系统操作

- ✓ **Controller（控制器）**：符合MVC架构的调用方式，Controller 处理来自表示层的请求，并向下调用系统操作
- ✓ **Service（业务逻辑）**：封装了系统操作，向下通过EntityManager对数据库中对象及其属性关系进行操作，从而完成系统操作
- ✓ **EntityManager（实体管理）**：管理对象及其属性关系，并进行持久化
- ✓ **Entity （实体类）**：提供Get/Set等基础操作，经过整合完成系统操作

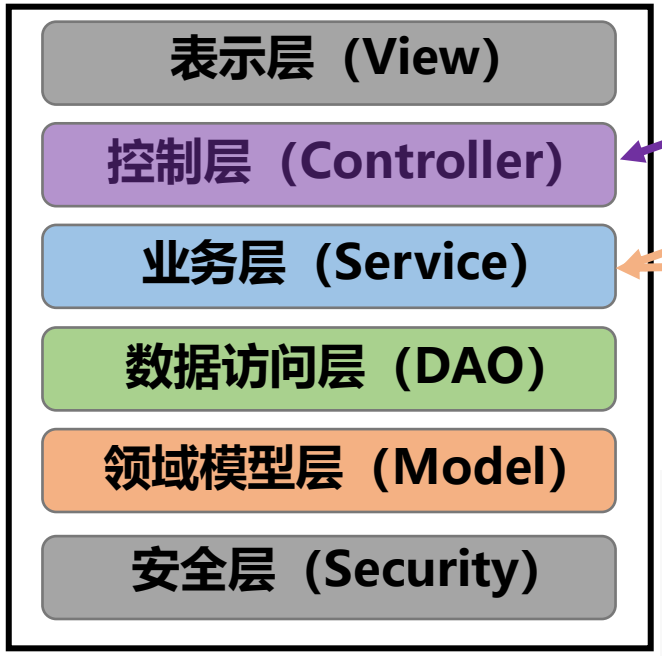


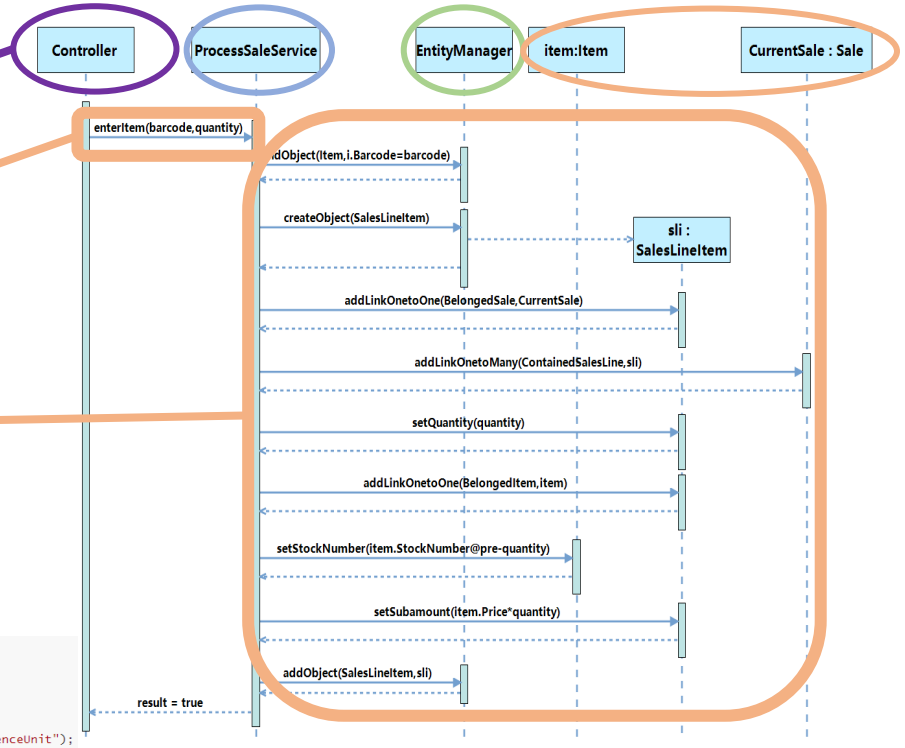
# 设计模型到Java EE代码

```java
import com.example.cocome.ProcessSaleService;
@WebServlet("/Controller")
public class Controller extends HttpServlet {
    // ...
    private void enterItem(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        int barcode = Integer.parseInt(request.getParameter("barcode"));
        int quantity = Integer.parseInt(request.getParameter("quantity"));

        ProcessSaleService processSaleService = new ProcessSaleService();
        processSaleService.enterItem(barcode, quantity);
    }
}
```

映射到控制层Controller的代码，包括向合适的服务类请求系统操作。




映射到业务层中方法的具体执行过程的代码。

```java
public class ProcessSaleService {

    private Sale currentSale;
    private SalesLineItem currentSaleLine;
    private PaymentMethod currentPaymentMethod;
    private static EntityManagerFactory emf = Persistence.createEntityManagerFactory("cocomePersistenceUnit");
    public boolean enterItem(int barcode, int quantity) {
        EntityManager em = emf.createEntityManager();
        Item item = em.find(Item.class, barcode);

        if (item != null && item.getStockNumber() > 0) {
            currentSaleLine = new SalesLineItem(item, quantity);
            currentSale.addSalesLineItem(currentSaleLine);
            item.setStockNumber(item.getStockNumber() - quantity);
            // ... 省略其他代码
            return true;
        }
        return false;
    }
    ...
}
```

# 设计模型到Java EE代码

类图中的实体类部分映射到领域模型层的代码。

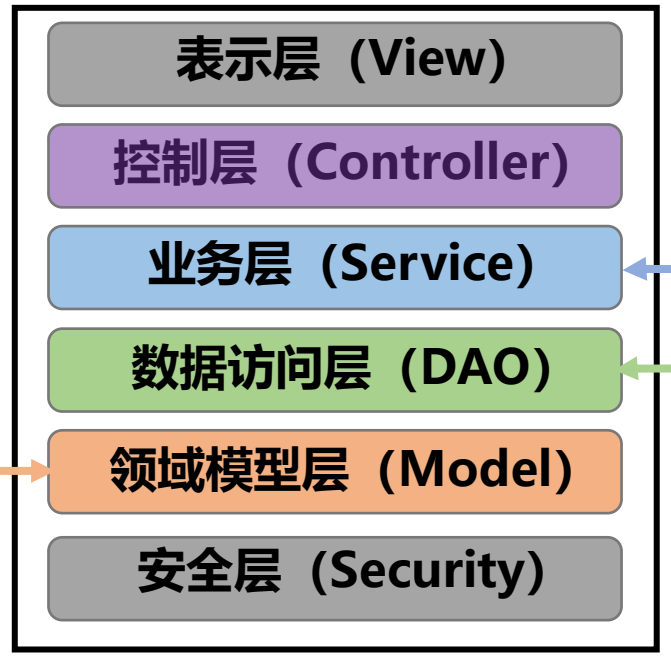
```java
package com.example.cocome.model;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class Item {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int barcode;
    private String name;
    private double price;
    private int stockNumber;
    private double orderPrice;

    public Item() {}

    // Getters and setters
}
```




类图中的服务类部分映射到业务层的代码。其中包括上层可以调用的方法、临时变量。

EntityManager映射到数据访问层JPA中使用的EntityManager。

```java
public class ProcessSaleService {

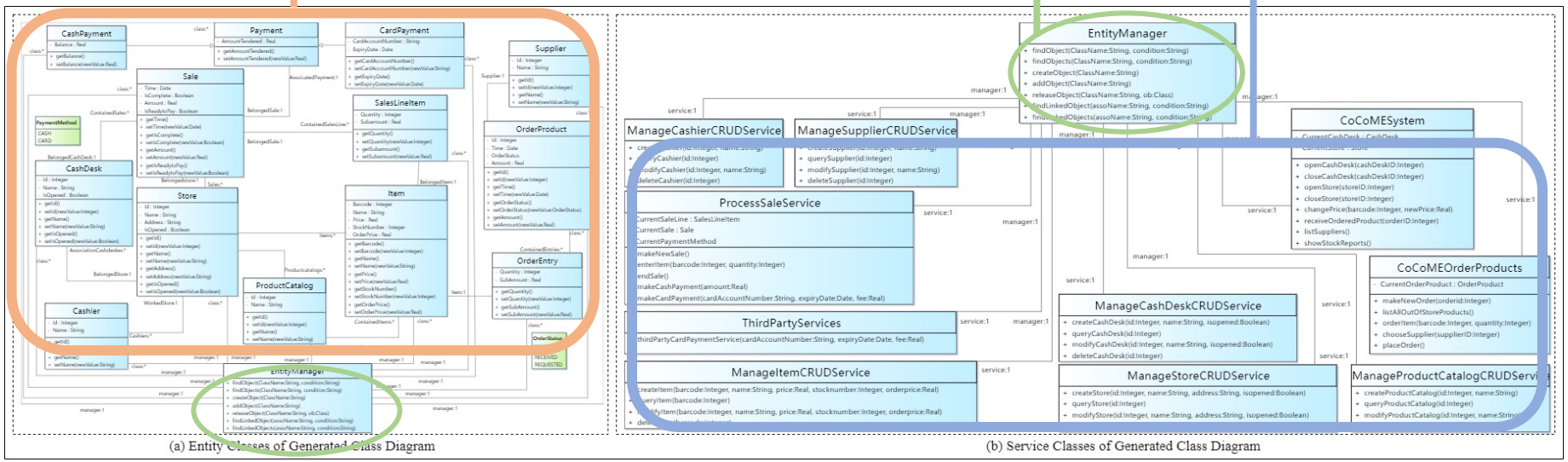
    private Sale currentSale;
    private SalesLineItem currentSaleLine;
    private PaymentMethod currentPaymentMethod;
    private static EntityManagerFactory emf = Persistence.createEntityManagerFactory("cocomePersistenceUnit");
    public void makeNewSale() {...}
    public boolean enterItem(int barcode, int quantity) {
        // 使用EntityManager查询Item
        EntityManager em = emf.createEntityManager();
        Item item = em.find(Item.class, barcode);

        if (item != null && !currentSale.getIsComplete() && item.getStockNumber() > 0) {
            // ... 省略其他代码
        } else {
            // ... 省略其他代码
        }
    }

    public void endSale() {...}

    public void makeCashPayment(BigDecimal amount) {...}

    public void makeCardPayment(String cardNumber, String cardHolderName, String expiryDate) {...}
}
```




(a) Entity Classes of Generated Class Diagram

(b) Service Classes of Generated Class Diagram

北京航空航天大學
BEIHANG UNIVERSITY



Part 4

# 软件文档

# 软件实现文档

- 软件实现文档由**源程序清单和用户指南**组成。由于面向对象的语言与面向过程的语言有所不同，所以完整的源程序清单有时打印不出来，往往只能打印出部分源程序清单。一般的做法是：不打印出源程序清单，只保留其电子版本。

- 用户指南主要包括**《用户使用手册》**和**《用户安装手册》**

- 在特殊情况下，例如，数据库服务器上的存储管理、数据维护、数据备份、数据传输等问题较复杂时，在用户指南中还要增加**《系统管理员手册》**的内容。

# 用户指南的编写要求

（1）统一采用编写工具Word 进行编写。

（2）**用统一的风格和确定的方法**，描述产品提供的所有功能。描述之后的功能，应该具有如下特点：对于**每一项功能的描述应该完整**。

（3）术语部分包括**系统术语和基本概念**。系统术语在整个系统的含义是统一的，放在前言部分描述。基本概念在各有关章节进行描述。

（4）**语言要简洁明快，表达准确**。一句话不要太长，中间可以断句几次。

（5）指南格式应统一。

# 用户指南的排版建议

(1) 2人以上合作编写，一定要有一人负责统稿。

(2) 交稿前一定要最后一次仔细审读。

(3) 图以章为序，如图5.1或图5-1（半字线连接章号和图号），图的标题在图的下方。

(4) 表以章为序，如表3.1或表3-2，表的标题在表的上方。

(5) 命令格式。操作步骤中使用的命令按钮、菜单命令项、标签项等名称用【】界定。如：打开【开始】菜单选择【程序】项……

(6) 例题格式。例题以章为序，并用【】界定。如【例3.5】或【例3-5】。

# 用户指南的排版建议

（7）交稿前一定要最后一次仔细审读，尽量减少不应有的文字录入错误，并做到5个衔接、11个统一、7个对应：

- 5个衔接是：章节号、表序号、图序号、公式号、页码号要连续衔接，不重、不漏。
- 11个统一是：格式、层次、名词术语、符号、代号、计量单位要统一、一致。
- 7个对应是：目录与正文标题、标题与内容、正文与插图、正文与表格、图中代号与图注、前后内容要对应，避免重复和矛盾。

（8）严禁用Word自动格式生成章节标题或序号。

## 用户指南的编写原则

(1) 列出应编制文档的目录。

(2) 提示编制文档应参考的标准。

(3) 指定文档管理员。

(4) 提供编制文档所需要的条件，落实文档编写人员、主管和所需经费以及编制工具等。

(5) 明确保证文档质量的方法，为了确保文档内容的正确性、合理性，应采取一定的措施，如评审、审稿、鉴定等。

(6) 绘制进度表，以图表形式列出各阶段应产生的文档、编制人员、编制主管、编制日期、完成日期、评审日期等。

# 软件实现文档(续)

## 用户指南的审核鉴定

为了保证文档的质量以及文档内容的正确性、合理性，软件工程管理部门应对用户指南进行评审和鉴定工作，提出改进意见，有利于整个文档质量的提高。

## 软件实现管理文档

软件实现管理文档包括《用户指南评审报告》，《模块源程序行统计表（行/模块名）》，《源程序工作量统计表（行/人天）》。这些管理文档均较简单。

# 小结

- 对于有经验的程序员来说，只要《详细设计说明书》设计合理、制作规范、评审通过（不符合项为零），软件实现是一件轻松愉快的事情。
- 若IT企业十分成熟，积累的软件财富丰厚，类库、构件库强大，规范和标准齐全，软件过程管理到位，那么软件实现更加简单。
- 按照本章介绍的软件实现原则、实现方法、实现管理、程序设计风格、屏幕界面控件设计、窗口级定义、系统级定义、帮助信息和用户指南编写等各种规范，就可以逐步实现《详细设计说明书》的内容。

# 小结

## ❑软件实现

- ✓软件实现包括编码、测试、调试、部署等一系列的活动
- ✓基于软件设计模型，编写出目标软件系统的程序代码，并对代码进行必要的测试，以发现和纠正代码存在中的缺陷，并将目标代码部署到计算机上运行

## ❑编程语言的选择

- ✓要根据软件所属的应用领域、与遗留软件系统的交互、程序员的经验等多个方面，考虑选择什么样的程序设计语言来进行编程

## ❑程序员遵循编码的原则和规范来编写出高质量的程序代码

北京航空航天大學
BEIHANG UNIVERSITY

# Thank you

北京航空航天大学

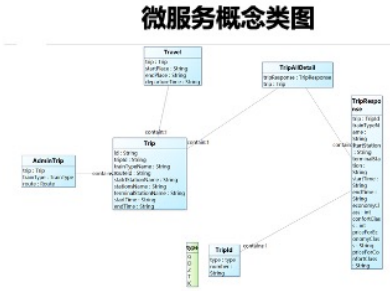
# 基于SpringCloud的微服务应用

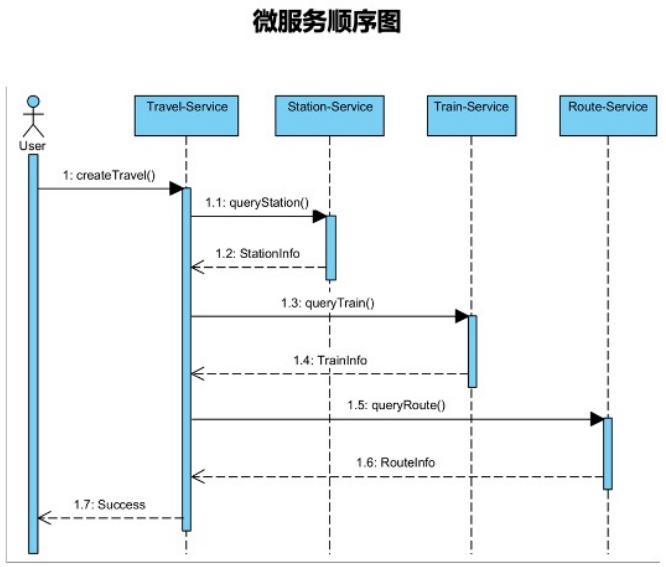
## 微服务设计模型

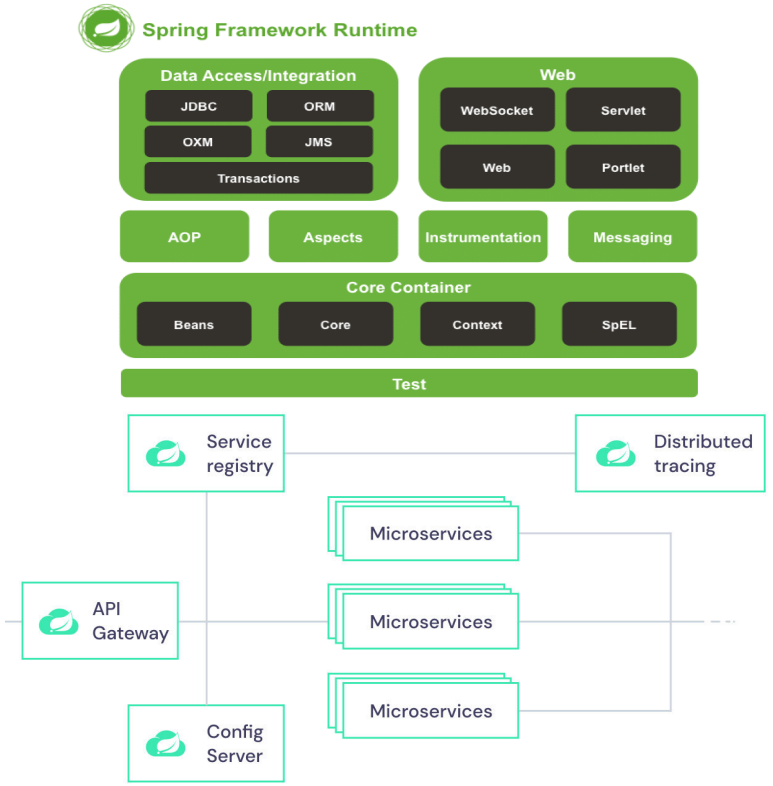
模型到架构映射过程

## SpringCloud框架



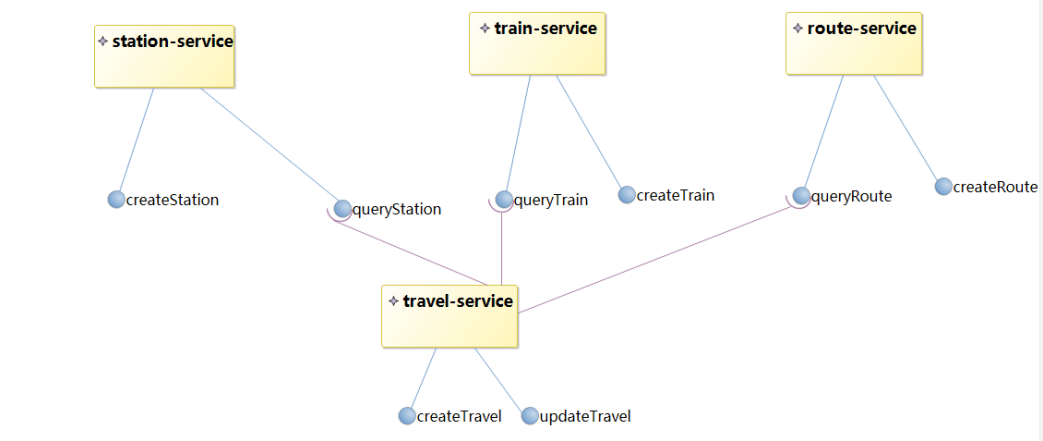

https://spring.io/projects/spring-cloud

# 微服务架构模型

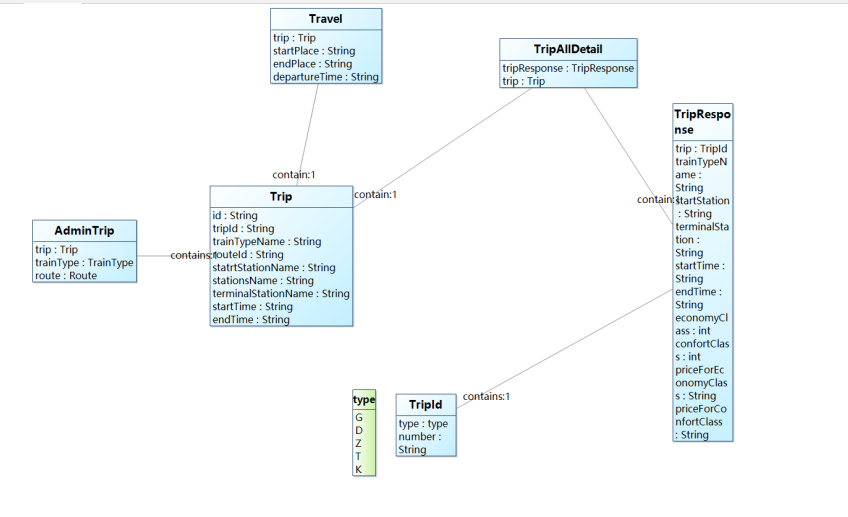
- **微服务架构图由微服务组件图、微服务内部概念类图、微服务顺序图组成**
  - ✓**微服务组件图**：说明了系统中的所有微服务及其相互依赖关系。它包含了每个微服务的内部类图，并划定了每个微服务提供和需要的接口，以及特定接口中的操作。
  - ✓**微服务概念类图**：描述每个微服务内部的实体以及实体之间的关系
  - ✓**微服务顺序图**：微服务序列图是服务之间互动的图形化表示，显示了服务之间的信息传递和互动的顺序。可以帮助开发者更好地理解服务之间的通信方式及其协议

# 微服务架构模型

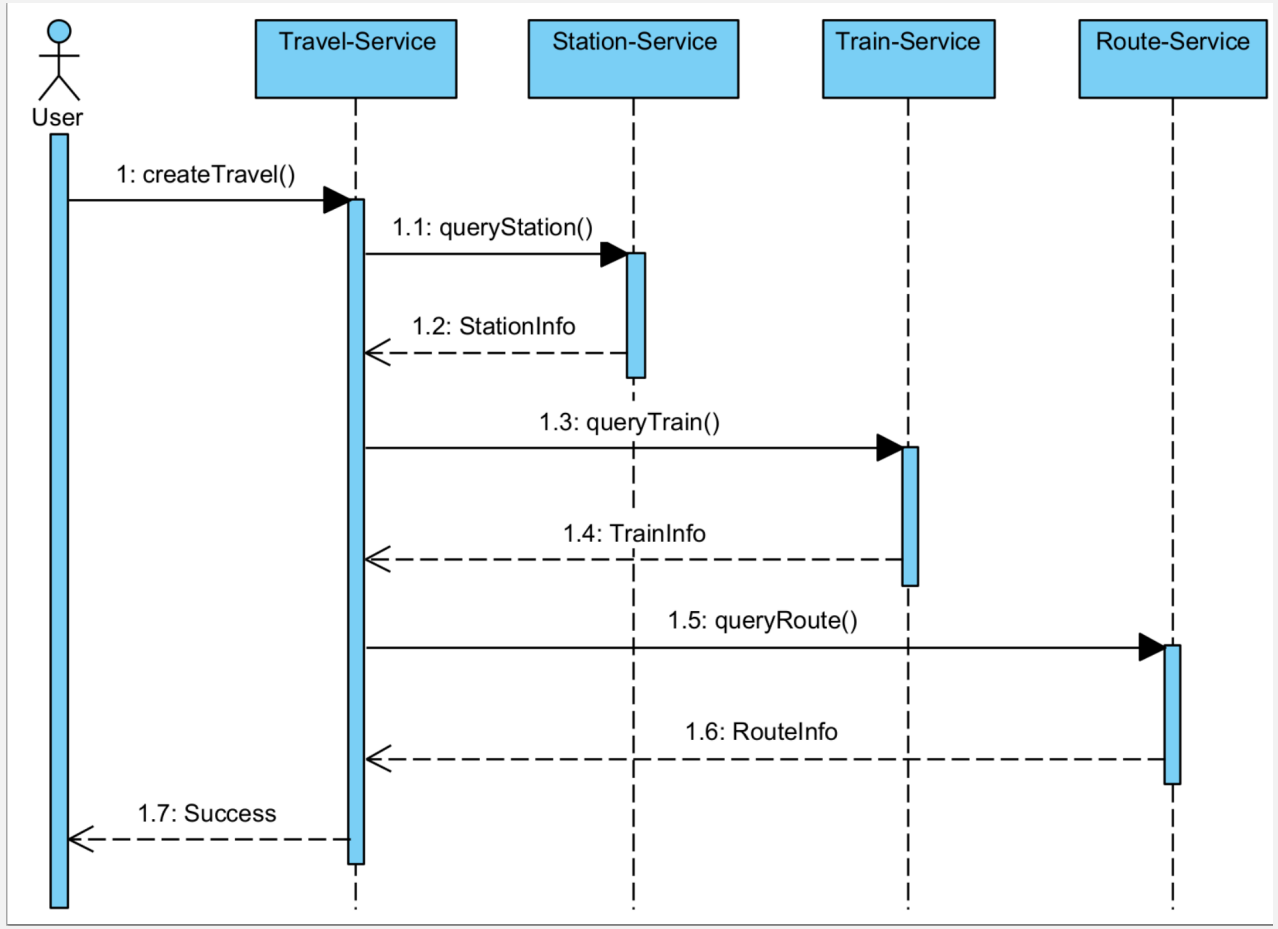
## 微服务组件图




## 微服务概念类图

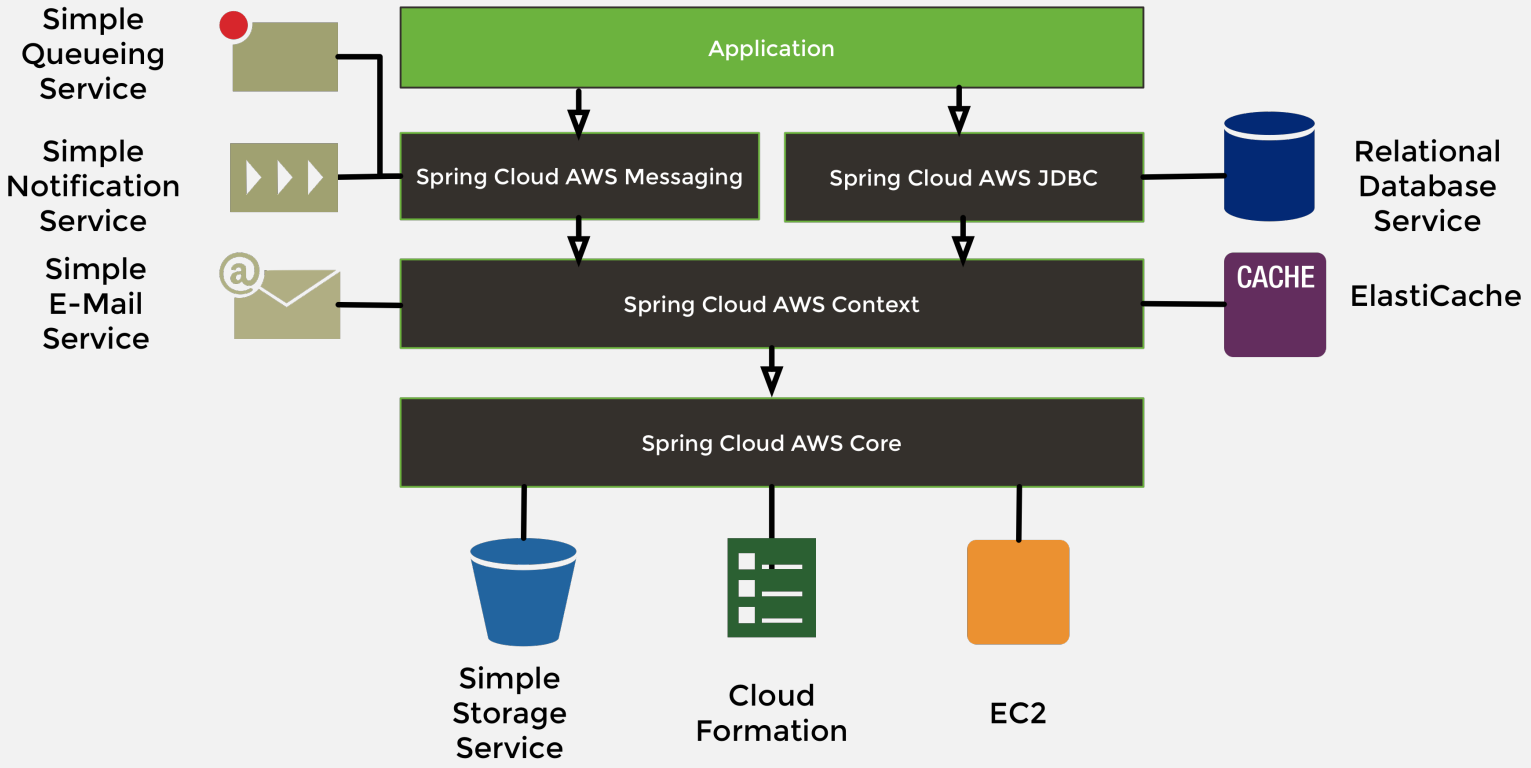



## 微服务顺序图



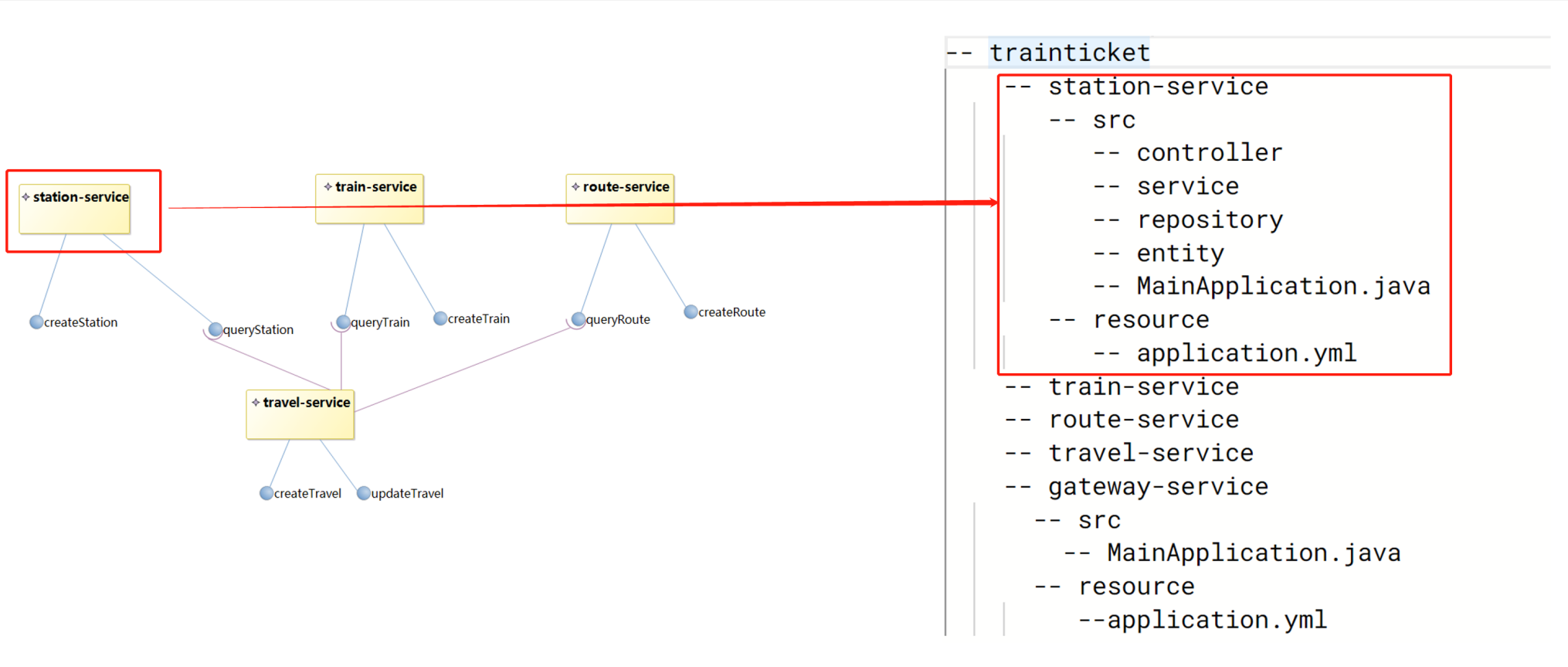
# Spring Cloud 微服务架构模型

- **Spring Cloud是一系列框架的有序集合，主要包括：**
  - ✓ 对现有成熟框架Spring Boot的封装和抽象
  - ✓ 实现了一部分分布式系统的基础设施功能，如Nacos扮演的就是服务注册和服务发现角色



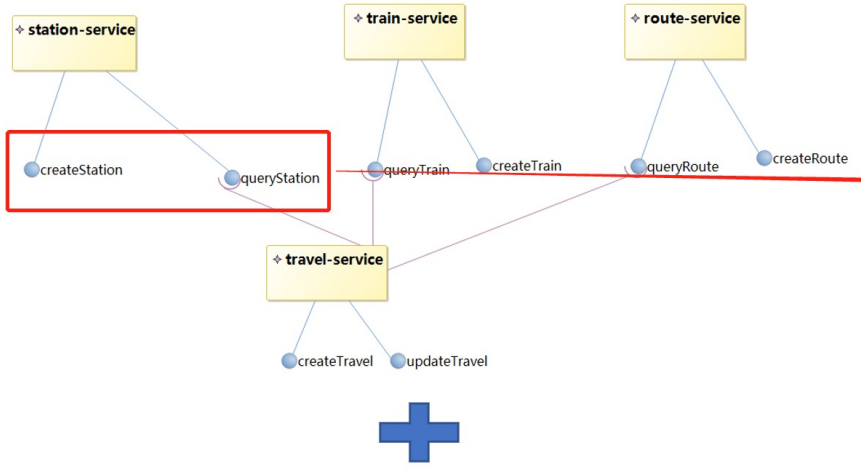
# Spring Cloud架构

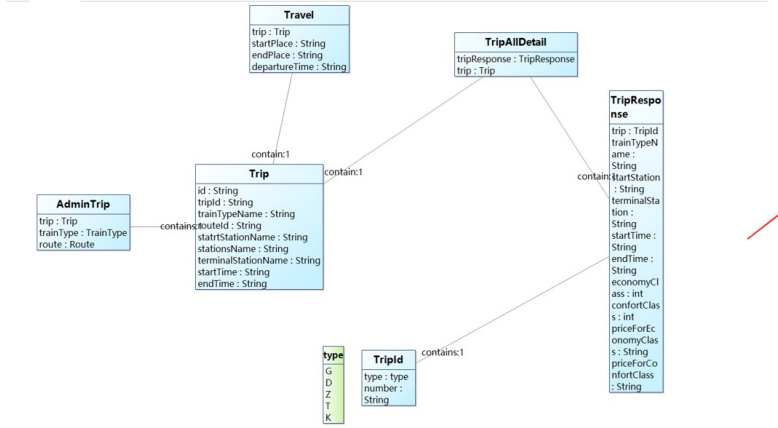
- 通过组件图可以搭建对应的SpringBoot项目，其中一个组件图就是一个SpringBoot项目



# Spring Cloud架构

- 通过组件图中的interface决定了controller层和service层，其中的概念类图决定了entity和repository层



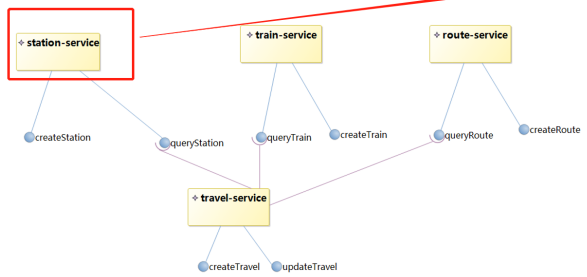




```
-- trainticket
    -- station-service
        -- src
            -- controller
            -- service
            -- repository
            -- entity
            -- MainApplication.java
        -- resource
            -- application.yml
    -- train-service
    -- route-service
    -- travel-service
    -- gateway-service
      -- src
        -- MainApplication.java
      -- resource
        --application.yml
```

文件目录结构

# 服务发现配置

- 服务发现通过application.yml文件同时在Main函数中加入注解@EnableDiscoveryClient,通过@LoadBalanced来实现负载均衡




```yaml
server:
  port: 9000
spring:
  application:
    name: stationservice
  cloud:
    nacos:
      discovery:
        server-addr: ${NACOS_ADDRS:127.0.0.1:8848}
    main:
      allow-bean-definition-overriding: true
# mysql
  datasource:
    url:
    username:
    password:
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQL5Dialect
```

注册在nacos服务名字

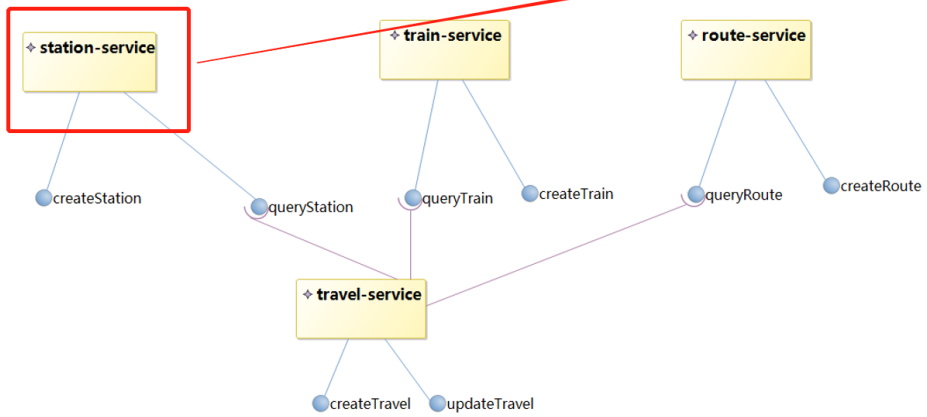
nacos地址

mysql配置信息

```java
@SpringBootApplication
@EnableAspectJAutoProxy(proxyTargetClass = true)
@EnableAsync
@IntegrationComponentScan
@EnableDiscoveryClient
public class StationApplication {
	public static void main(String[] args) {
		SpringApplication.run(StationApplication.class, args);
	}
	@LoadBalanced
	@Bean
	public RestTemplate restTemplate(RestTemplateBuilder builder) {
		return builder.build();
	}
}
```

# 网关实现

## ❑微服务需要网关的代码来管理api的转发过程




gateway application.yml

```
routes:
 -id : StationService
  uri : lb://${ADMIN_BASIC_INFO_SERVICE_HOST:StationService}
  predicates:
    - Path = /stationservice/**
 -id : TrainService
  uri : lb://${ADMIN_BASIC_INFO_SERVICE_HOST:TrainService}
  predicates:
    - Path = /trainservice/**
 -id : RouteService
  uri : lb://${ADMIN_BASIC_INFO_SERVICE_HOST:RouteService}
  predicates:
    - Path = /routeservice/**
 -id : PriceService
  uri : lb://${ADMIN_BASIC_INFO_SERVICE_HOST:PriceService}
  predicates:
    - Path = /priceservice/**
 -id : BasicService
  uri : lb://${ADMIN_BASIC_INFO_SERVICE_HOST:BasicService}
  predicates:
    - Path = /basicservice/**
 -id : TravelService
  uri : lb://${ADMIN_BASIC_INFO_SERVICE_HOST:TravelService}
  predicates:
    - Path = /travelservice/**
```

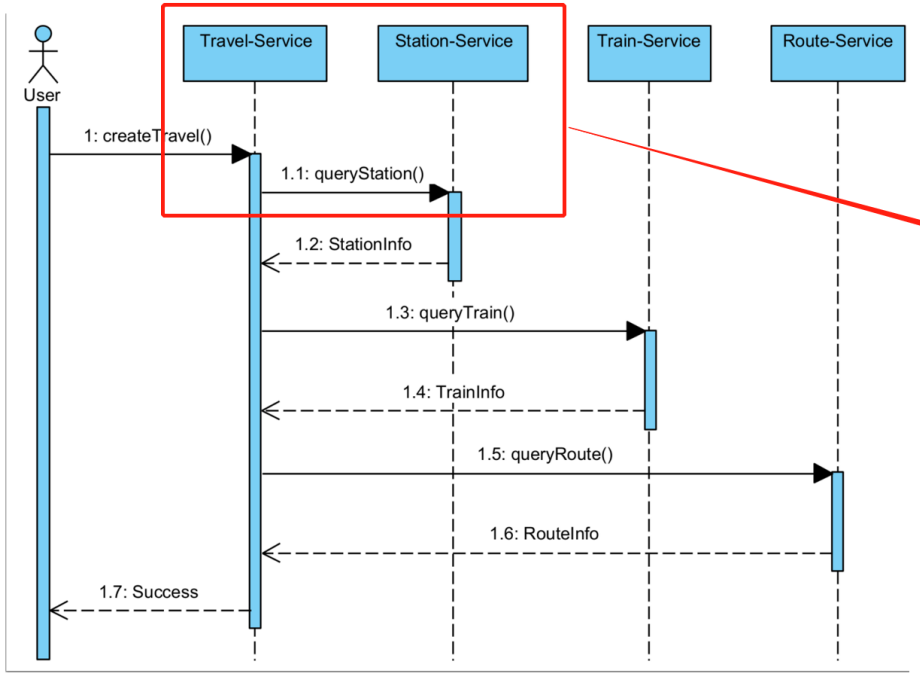
# 网关实现

- Gateway服务通过配置application.yml，routes中通过断言来实现api的转发

```yaml
server:
  port: 10008
spring:
  application:
    name: gateway
  cloud:
    nacos:
      discovery:
        server-addr: ${NACOS_ADDRS:127.0.0.1:8848}
    sentinel:
      transport:
        dashboard: ${SENTINEL_ADDRS:127.0.0.1:8888}
    gateway:
      discovery:
        locator:
          enabled: false
      routes:
        -id : StationService
         uri : lb://${ADMIN_BASIC_INFO_SERVICE_HOST:StationService}
         predicates:
          - Path = /stationservice/**
```

# 微服务顺序图

- 微服务顺序图表示显示了服务之间的信息传递和互动的顺序，在service层描述操作的顺序，通过restTemplate组件来实现




```
HttpEntity requestEntity = new HttpEntity(null);
String route_service_url = "http://ts-station-service";
ResponseEntity<Response> re = restTemplate.exchange(
        route_service_url + "/api/v1/routeservice/station/" +
        stationId,  HttpMethod.GET,  requestEntity,
        Response.class);
Response routeRes = re.getBody();
```

# 例子：Spring Cloud 应用在阿里云部署

